

Graph Representation Learning

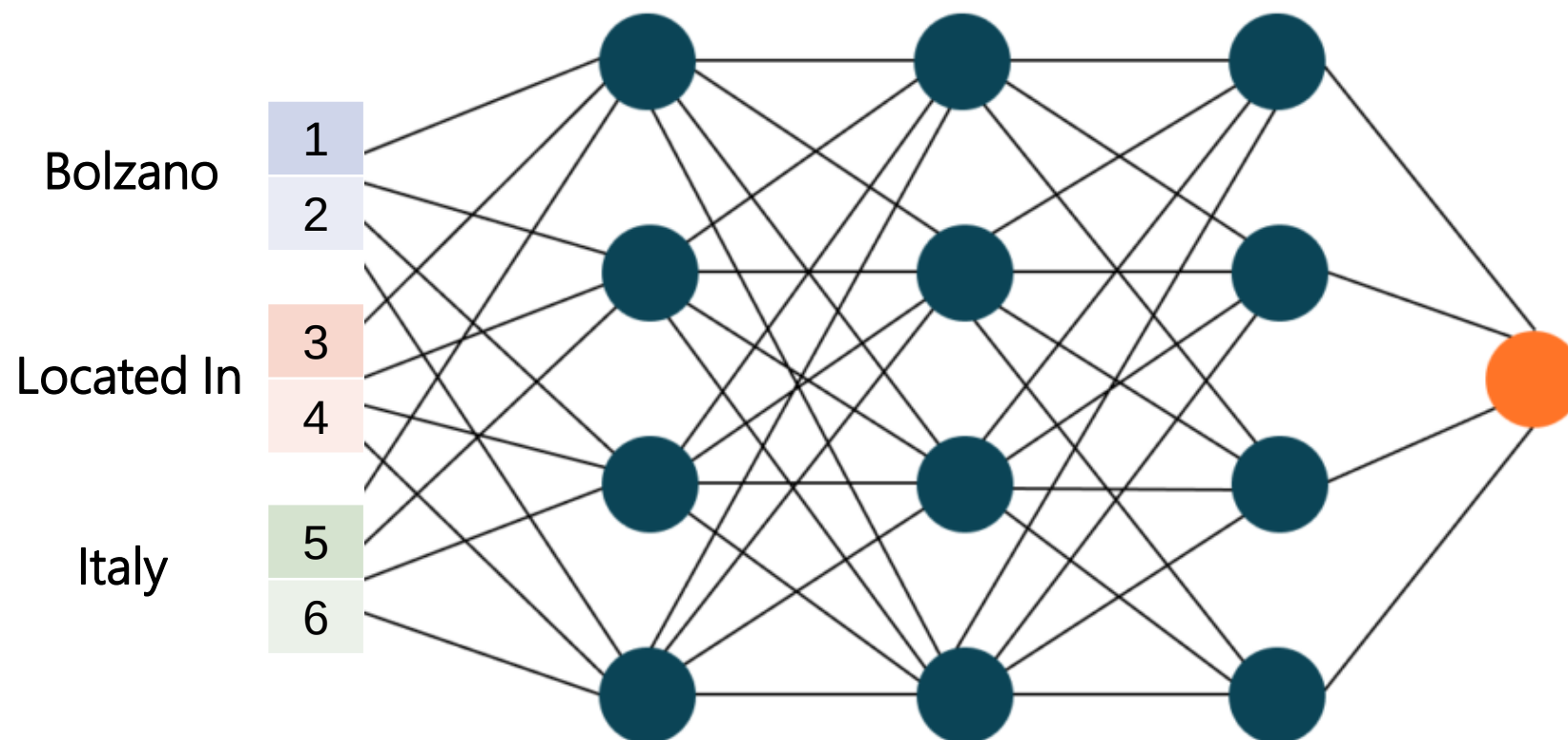
MDS

Graph Neural Networks

Giving more context to the embeddings

GNN: Introduction

We have seen that KGE Models can leverage **deep learning architectures**, but they still operate at the **triple level**.



GNN: Introduction

Instead of just focusing on **node-edge-node** (e.g., triple) level, can we explicitly leverage the **local** or even **global** graph structure?

Bolzano

1
2

Located In

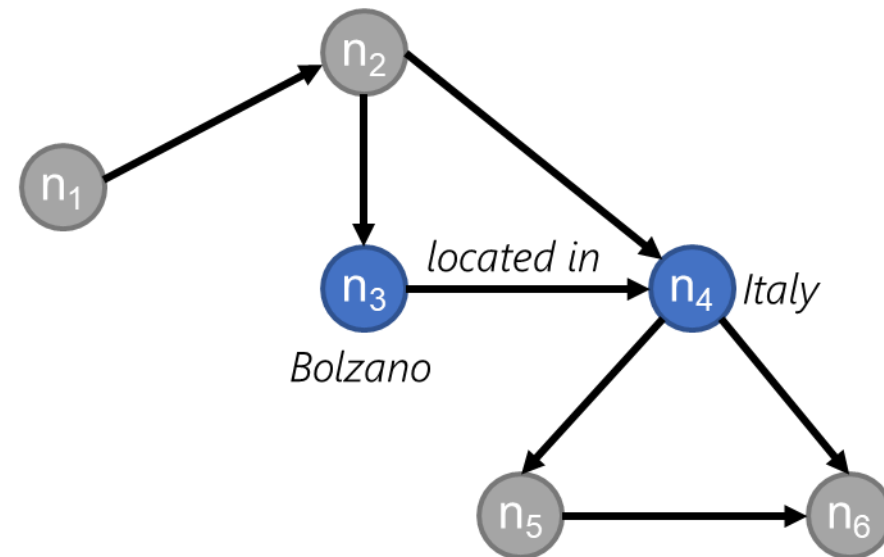
3
4

Italy

5
6

(Bolzano, Located In, Italy)

$$\begin{bmatrix} 1 & 2 \end{bmatrix} \times \begin{bmatrix} 3 & 0 \\ 0 & 4 \end{bmatrix} \times \begin{bmatrix} 5 \\ 6 \end{bmatrix} = 63$$



GNN: Introduction

Instead of just focusing on **node-edge-node** (e.g., triple) level, can we explicitly leverage the **local** or even **global** graph structure?

Bolzano

1
2

Located In

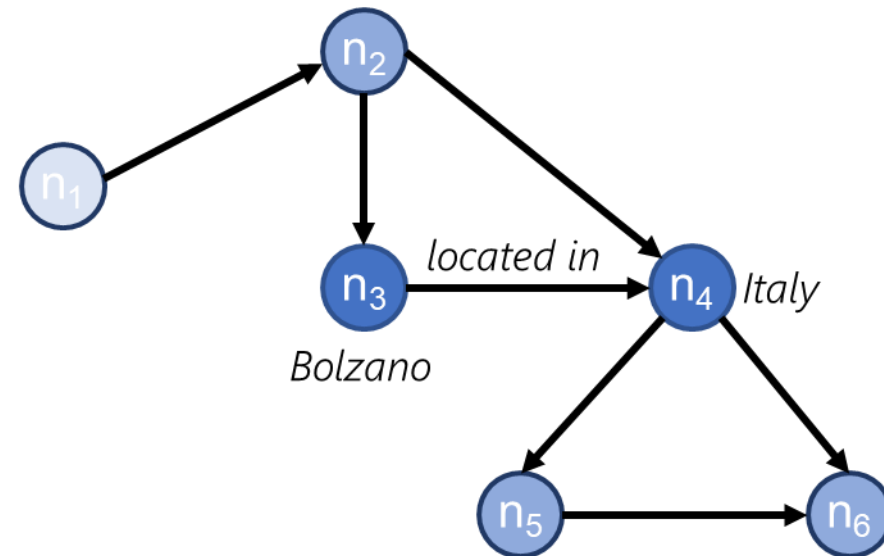
3
4

Italy

5
6

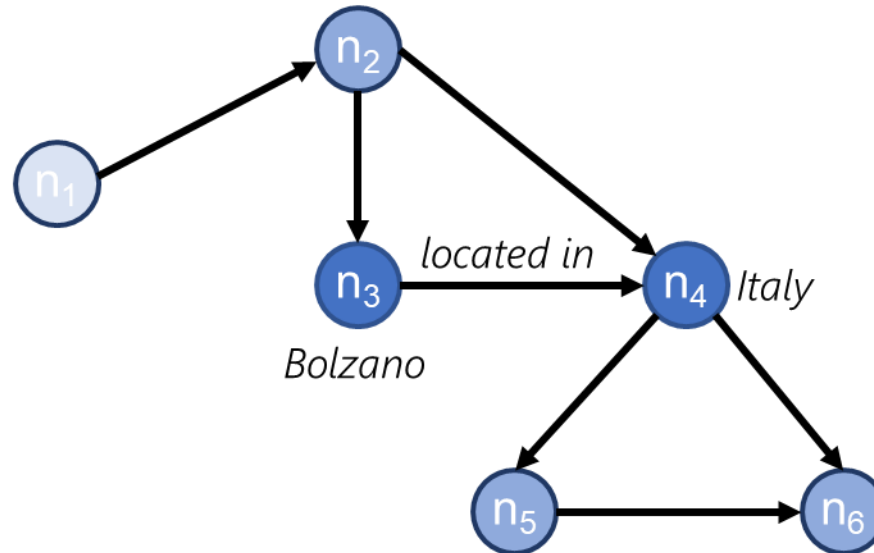
(Bolzano, Located In, Italy)

$$\begin{bmatrix} 1 & 2 \end{bmatrix} \times \begin{bmatrix} 3 & 0 \\ 0 & 4 \end{bmatrix} \times \begin{bmatrix} 5 \\ 6 \end{bmatrix} = 63$$



GNN: Introduction

Given the superior results **deep learning** has shown in processing text, images and other data, it has also been applied to **graphs**, originating the **Graph Neural Network** field.



GNN: Introduction

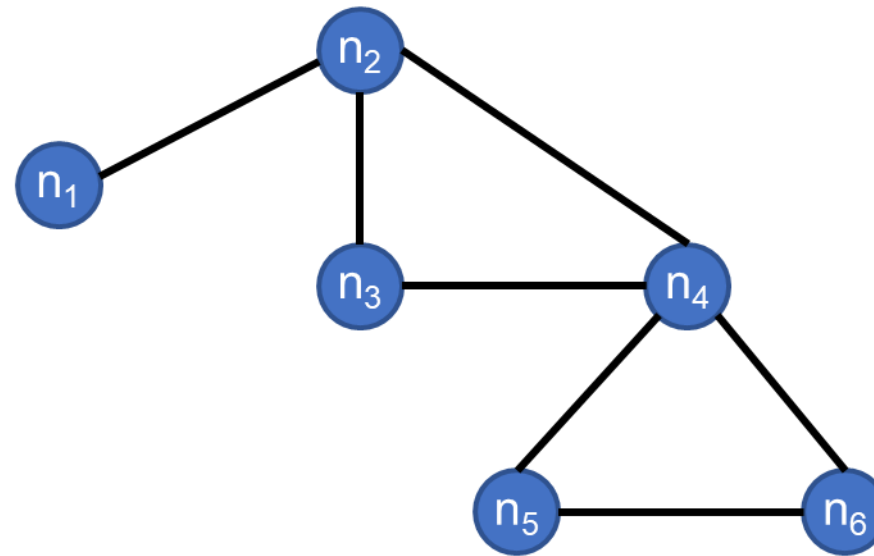
However, applying deep learning to graphs is **not trivial**:

1. **Irregular Structure**: it makes it difficult to produce **generalized** algorithms , unlike images (grid) or text (sequences). seq of words can be defined of length n, in images length nxn
2. **Heterogeneity**: graphs are diverse, some having **properties**, node **types**, etc. that should be leveraged.
3. **Scalability**: graphs could have **millions** of entities and **billions** of triples (e.g., Wikidata).

GNN: The basics

To apply GNNs, we need a **graph**, node/edge **features** and a **task** to solve:

Graph:

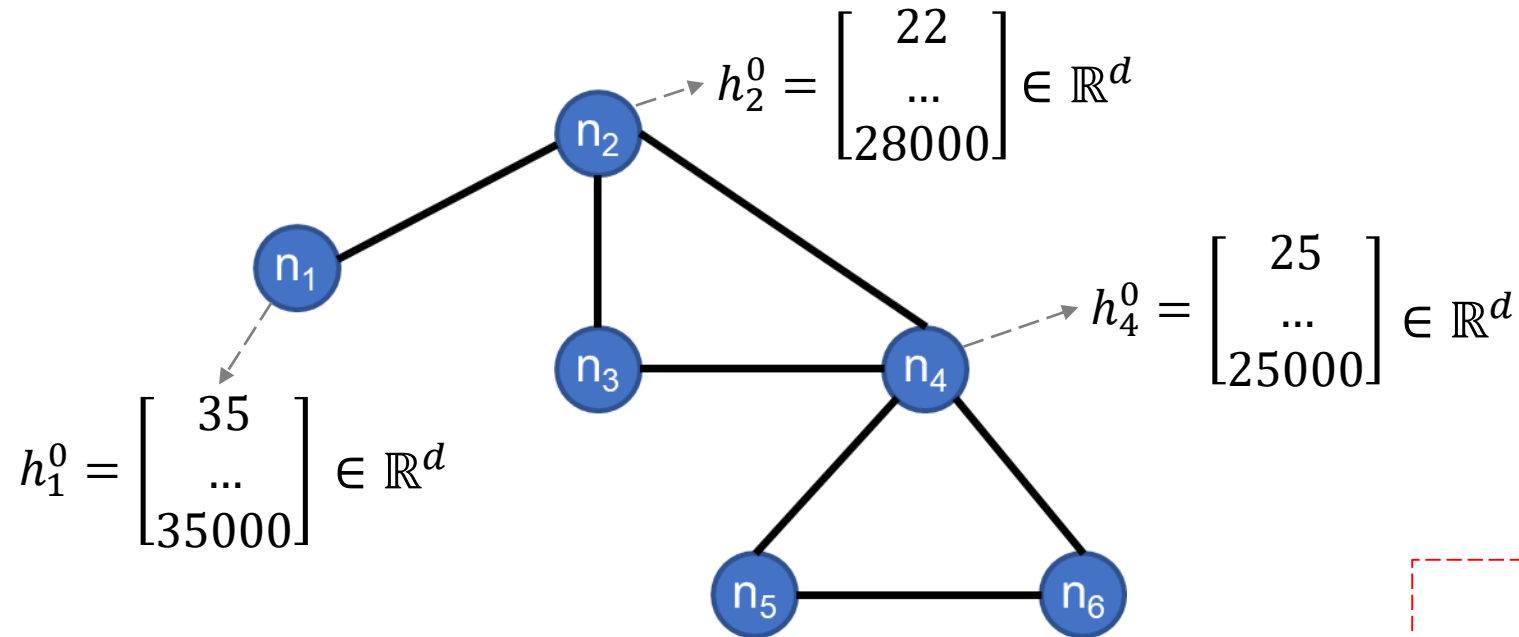


$$G = (V, E)$$

GNN: The basics

To apply GNNs, we need a **graph**, node/edge **features** and a **task** to solve:

Features:



we give one representation that will propagate during the ggraph

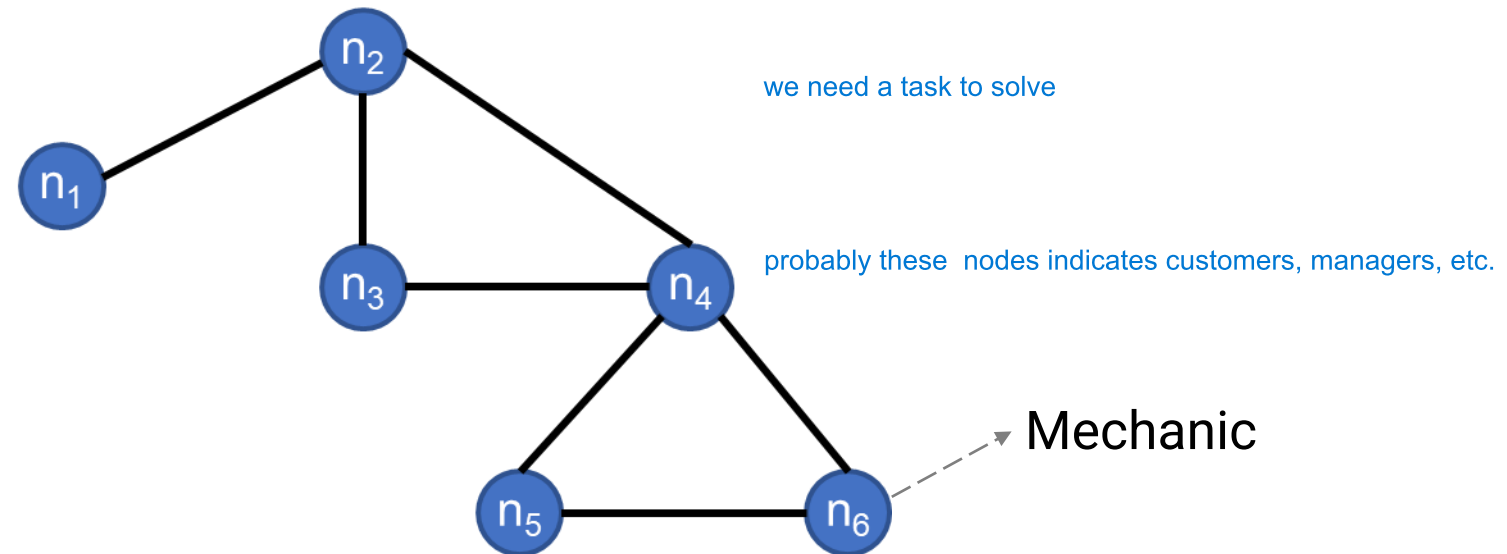
More on that later!

$$G = (V, E)$$

GNN: The basics

To apply GNNs, we need a **graph**, node/edge **features** and a **task** to solve:

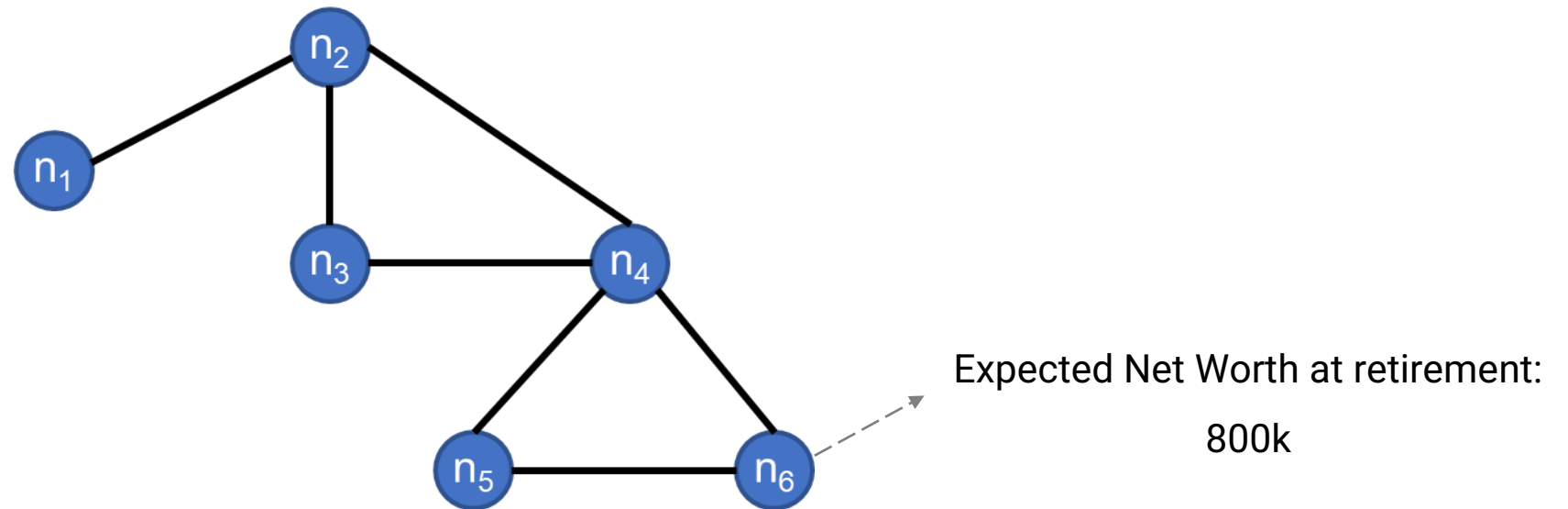
Task: Node classification



GNN: The basics

To apply GNNs, we need a **graph**, node/edge **features** and a **task** to solve:

Task: Node Regression

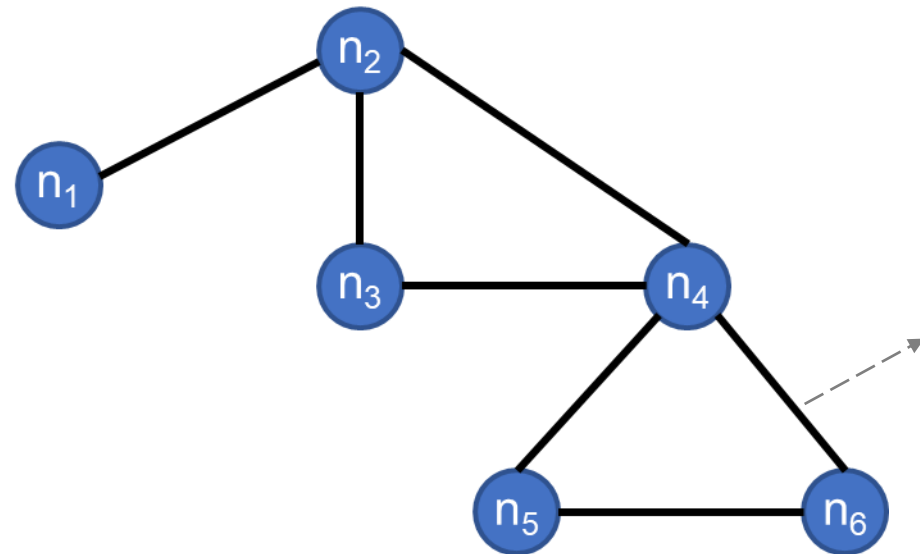


GNN: The basics

To apply GNNs, we need a **graph**, node/edge **features** and a **task** to solve:

we can go

Task: Edge Classification

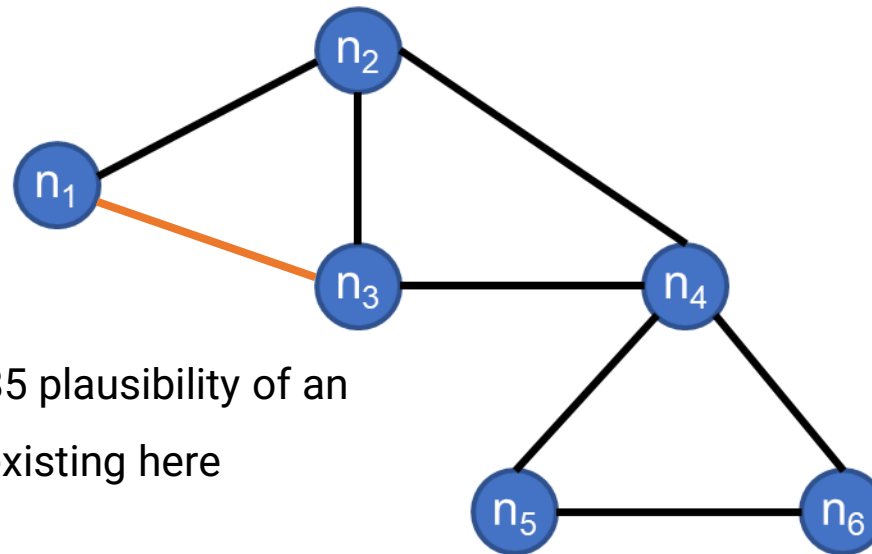


This relation indicates **customer**.

GNN: The basics

To apply GNNs, we need a **graph**, node/edge **features** and a **task** to solve:

Task: Edge Regression



There is a 0.85 plausibility of an edge existing here

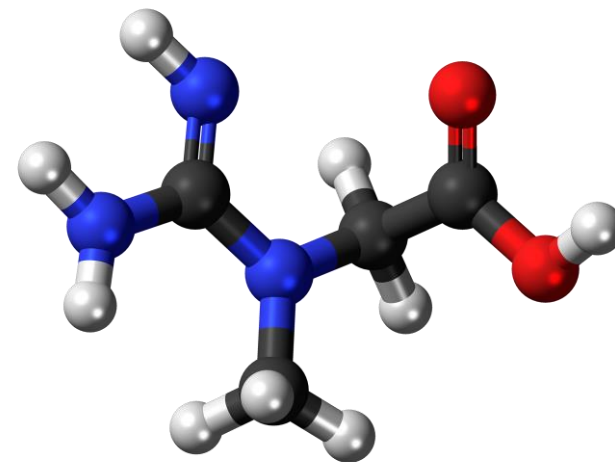
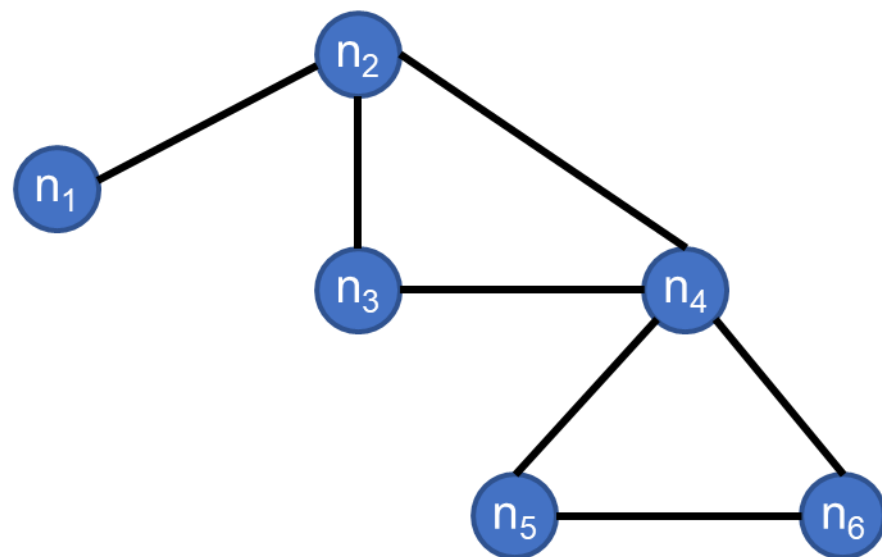
**The Link Prediction task
we saw with KGE**

GNN: The basics

To apply GNNs, we need a **graph**, node/edge **features** and a **task** to solve:

Task: Graph Classification

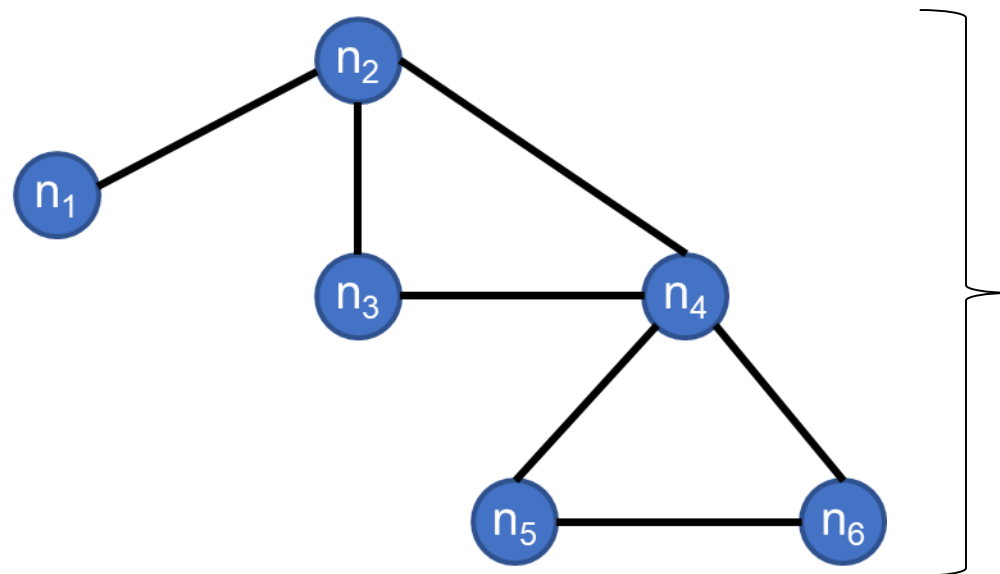
we can go from node level to graph level



GNN: The basics

To apply GNNs, we need a **graph**, node/edge **features** and a **task** to solve:

Task: Graph Classification

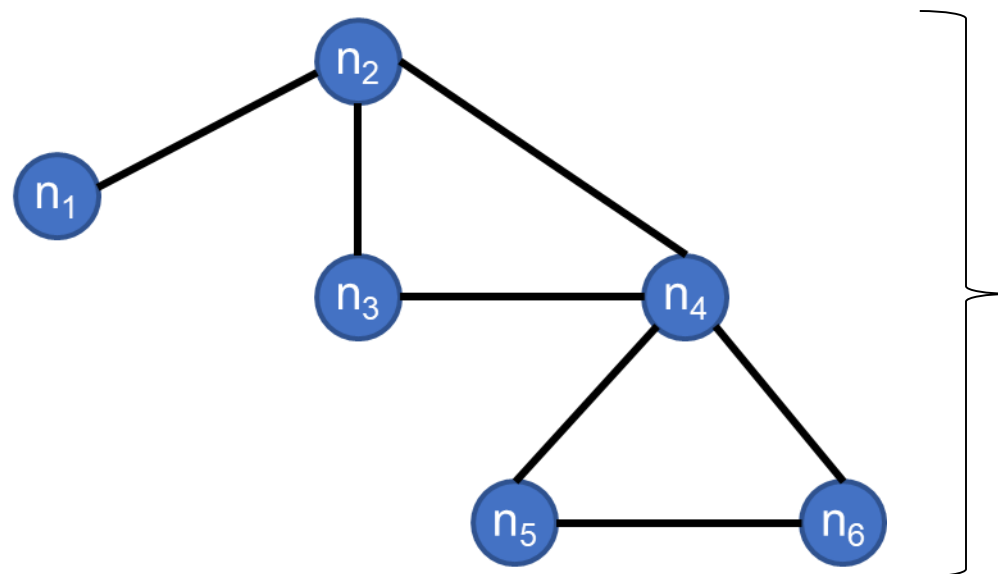


This **molecule** is hydrophobic.

GNN: The basics

To apply GNNs, we need a **graph**, node/edge **features** and a **task** to solve:

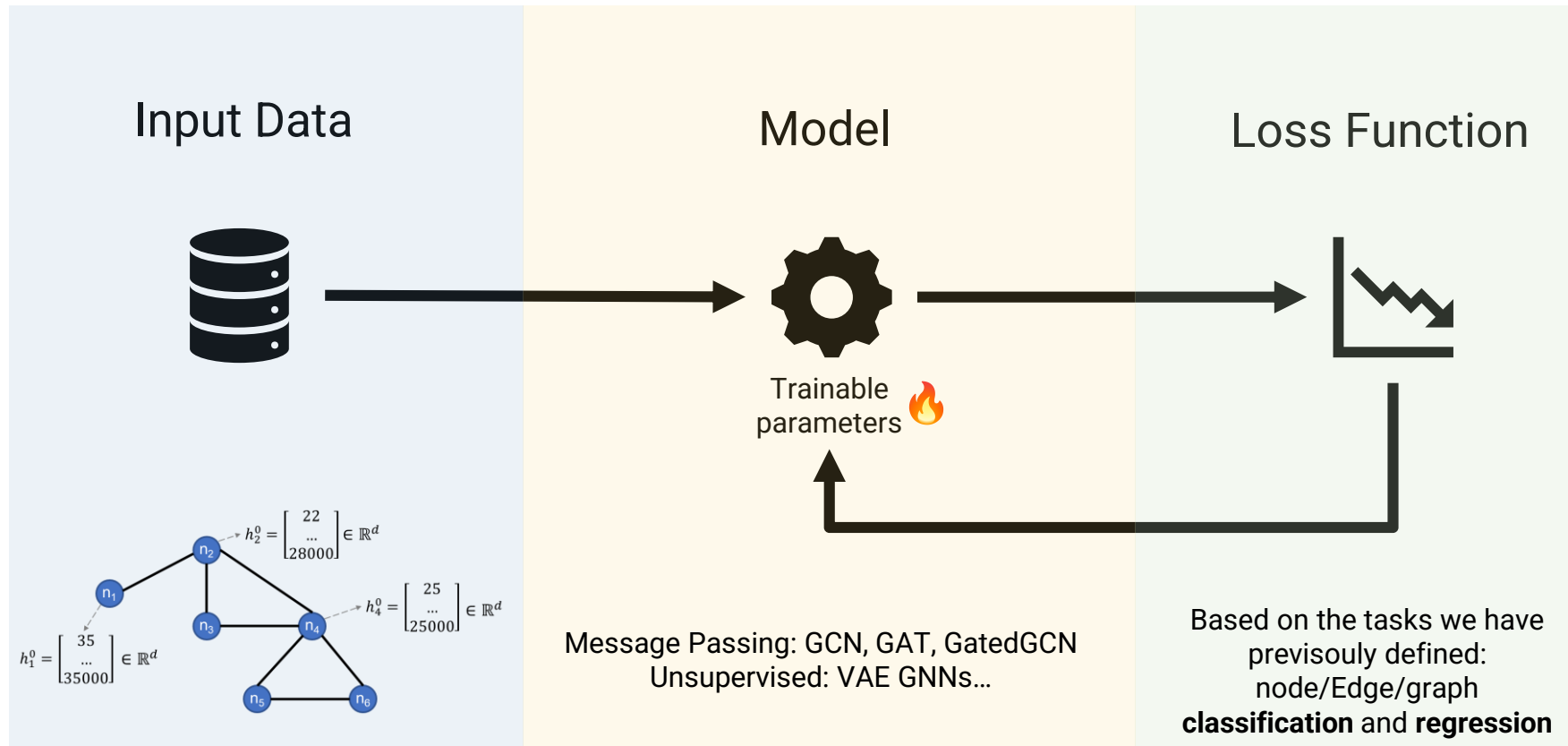
Task: Graph Regression



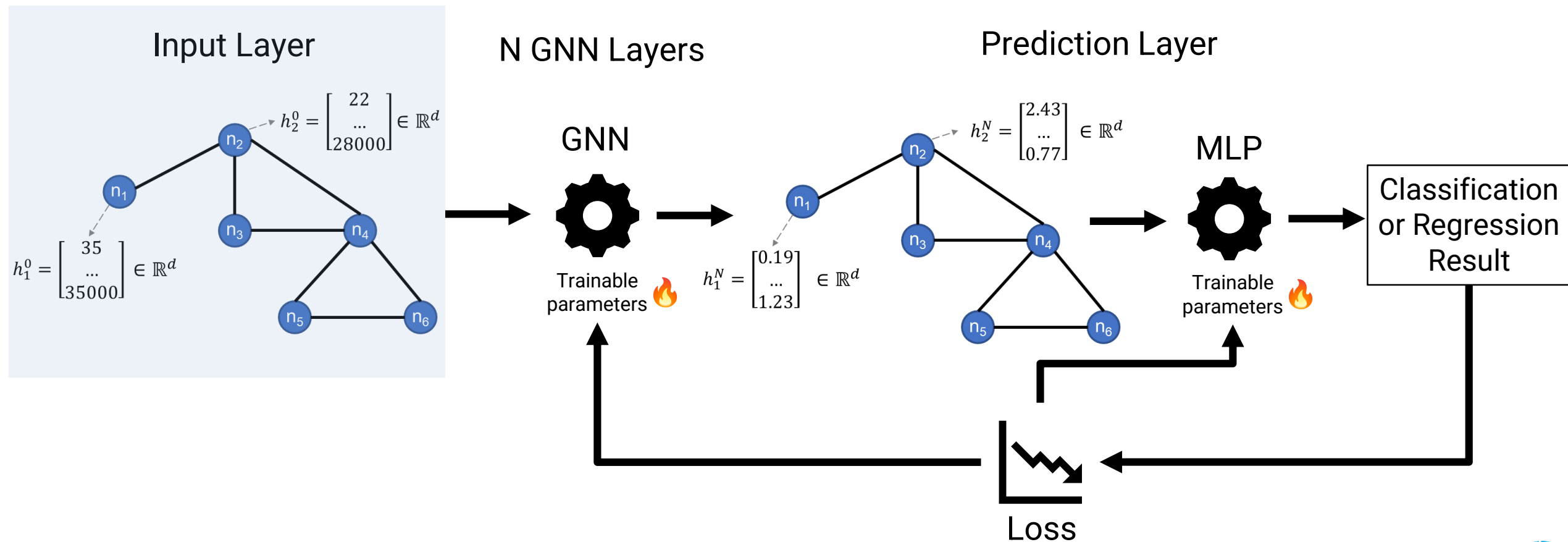
This **molecule** has molecular weight of 220g/mol.

Deep Learning: Basic Recipe

Typically, to train a **Deep Learning model** we need:

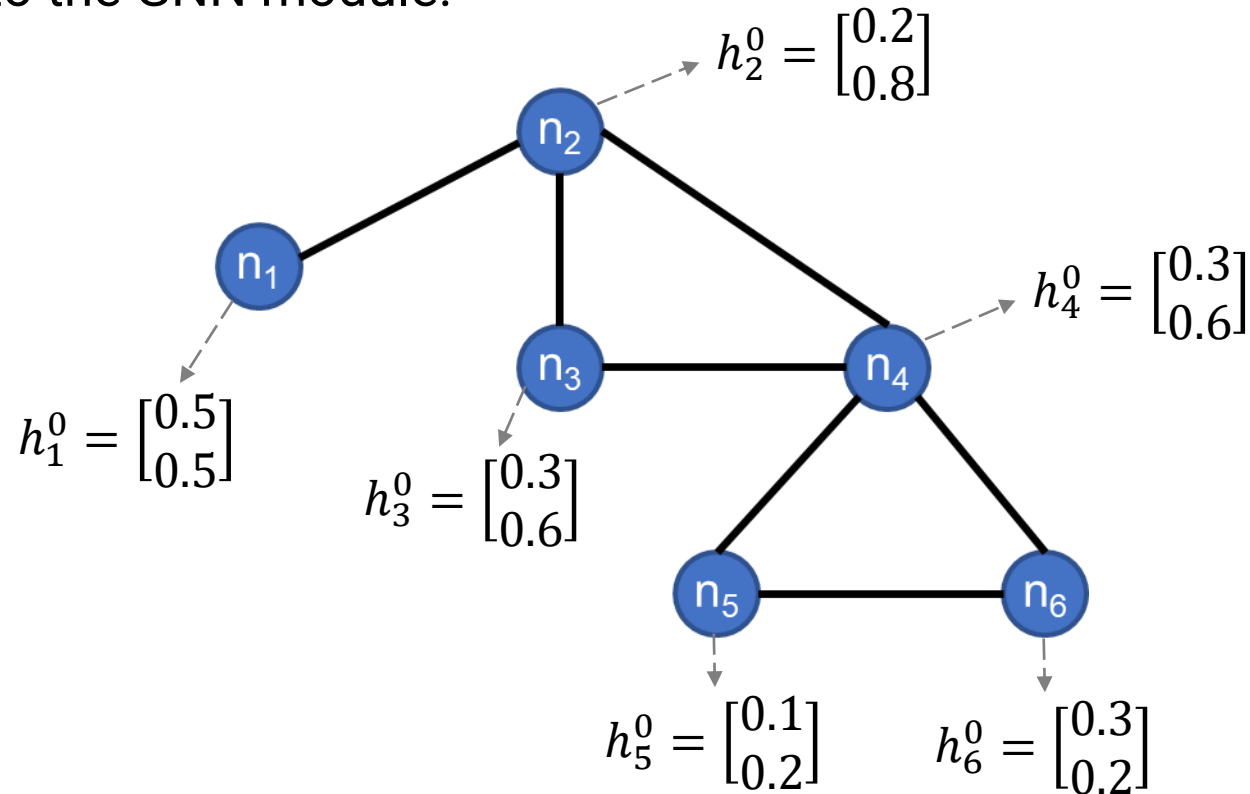


GNN: Basic Recipe



GNN: Input Layer

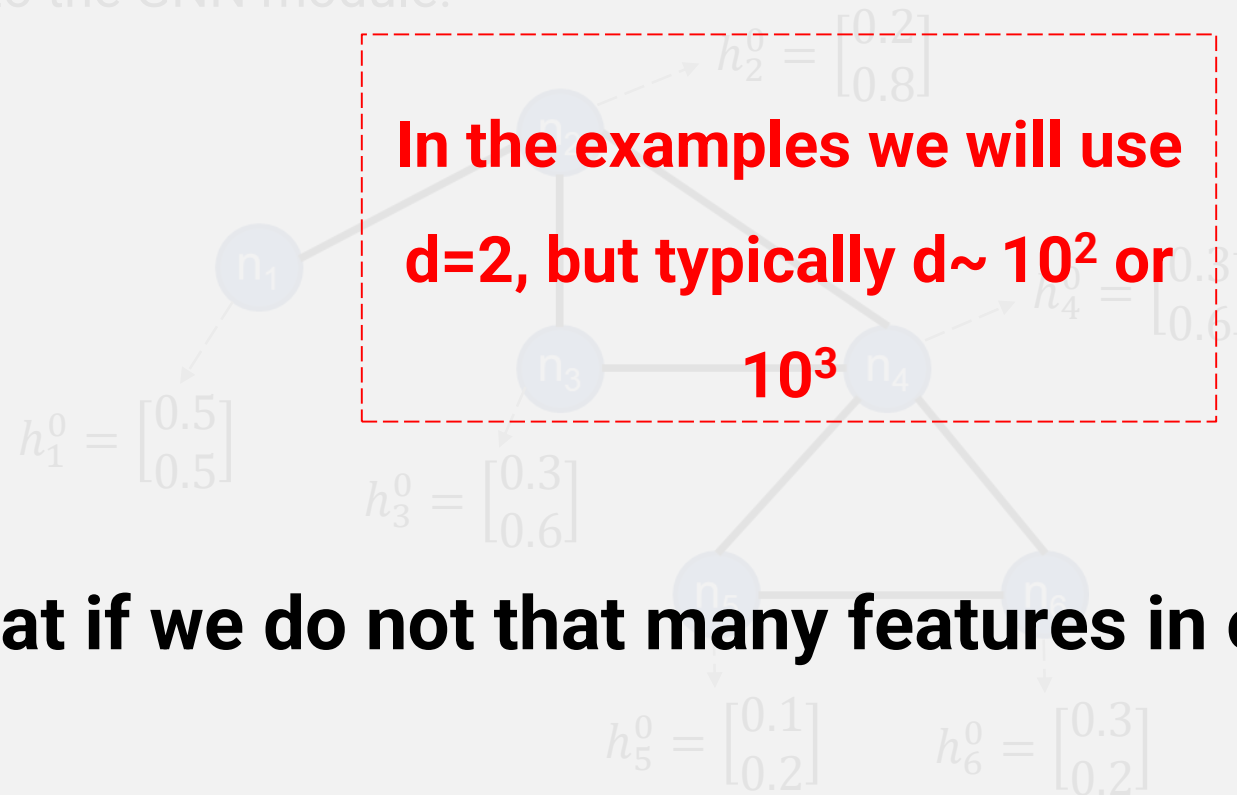
In the input layer, we represent each node/edge by its **features**, generating the vectors that are going to be **fed** to the GNN module.



In the examples we will use $d=2$, but typically $d \sim 10^2$ or 10^3

GNN: Input Layer

In the input layer, we represent each node/edge by its **features**, generating the vectors that are going to be **fed** to the GNN module.



What if we do not that many features in our nodes?

GNN: Input Layer

a veces no tenemos tantos órdenes de magnitud de las características de entrada. Si estamos trabajando con un vector de dim 2, y queremos obtener un vector de dim 4. Necesitamos aprender una matriz para ir de una dimensión a la otra dim, en este caso la dimensión se llama W.

We can use **learnable** linear projections to obtain a new vector of our **desired dimension**:

HJA 21/5

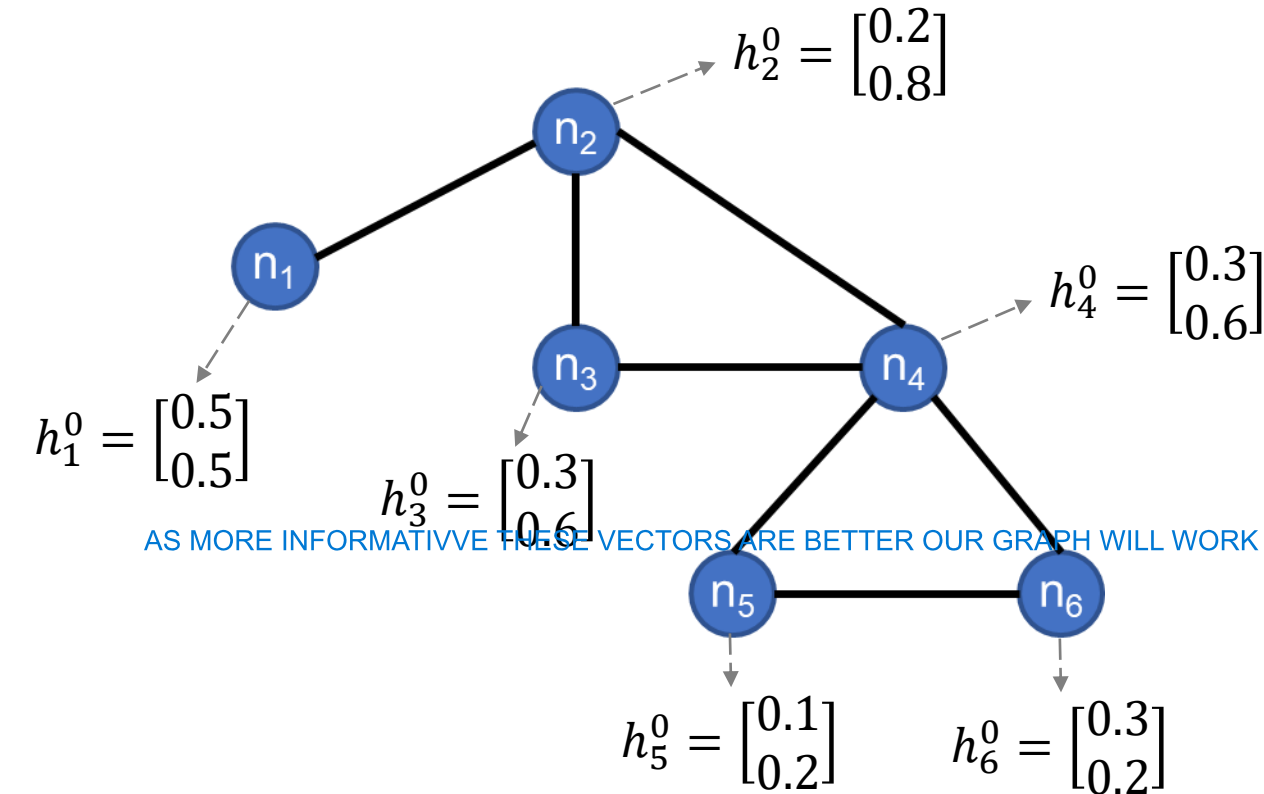
$$h_i^0 \in \mathbb{R}^2 \quad \longrightarrow \quad h_i'^0 \in \mathbb{R}^4$$

$$W \in \mathbb{R}^{4 \times 2}$$

$$h_i'^0 = W h_i^0$$

GNN: Input Layer

We can use **learnable** linear projections to obtain a new vector of our **desired dimension**:

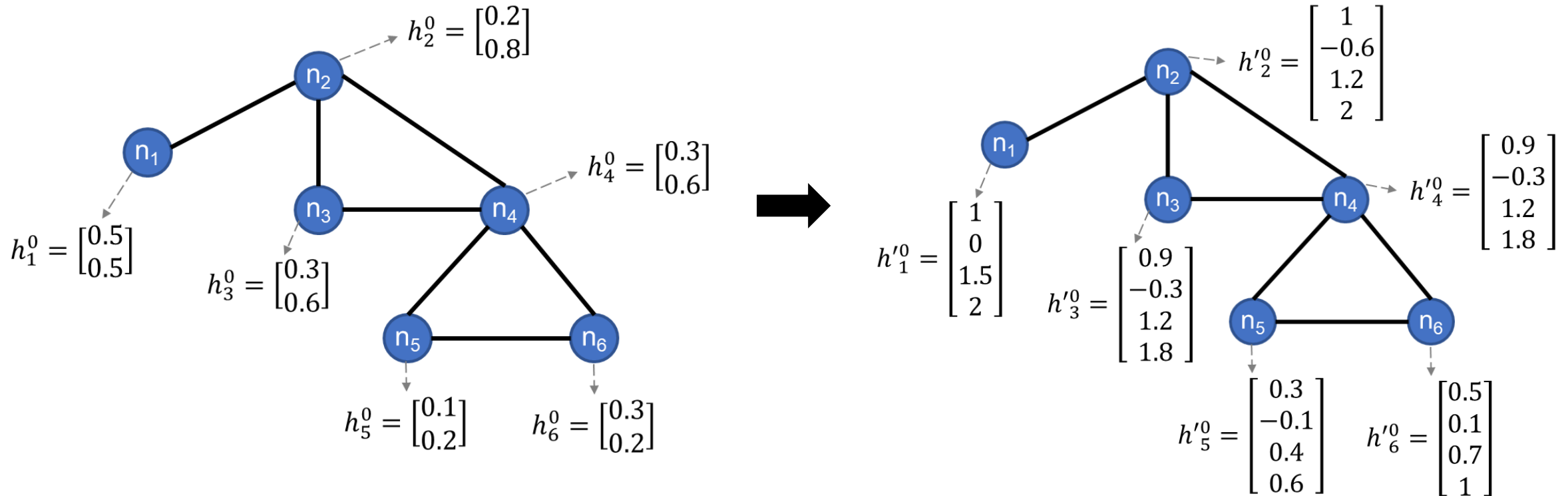


$$h_5^0 = \begin{bmatrix} 0.1 \\ 0.2 \end{bmatrix} \quad W = \begin{pmatrix} 1 & 0.5 \\ 4 & 2 \\ 1 & 0.5 \\ 1 & 2 \end{pmatrix}$$

$$h_5'^0 = \begin{bmatrix} 0.2 \\ 0.8 \\ 0.2 \\ 0.5 \end{bmatrix}$$

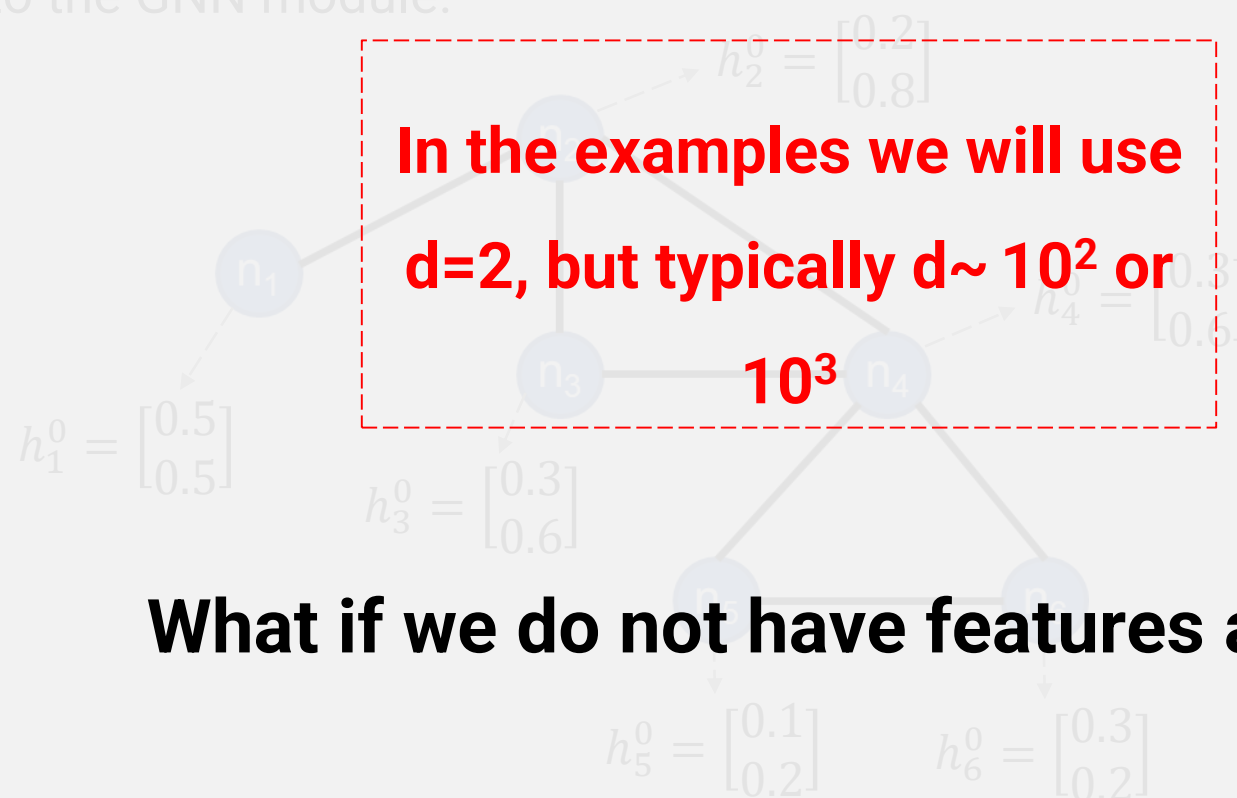
GNN: Input Layer

We can use **learnable** linear projections to obtain a new vector of our **desired dimension**:



GNN: Input Layer

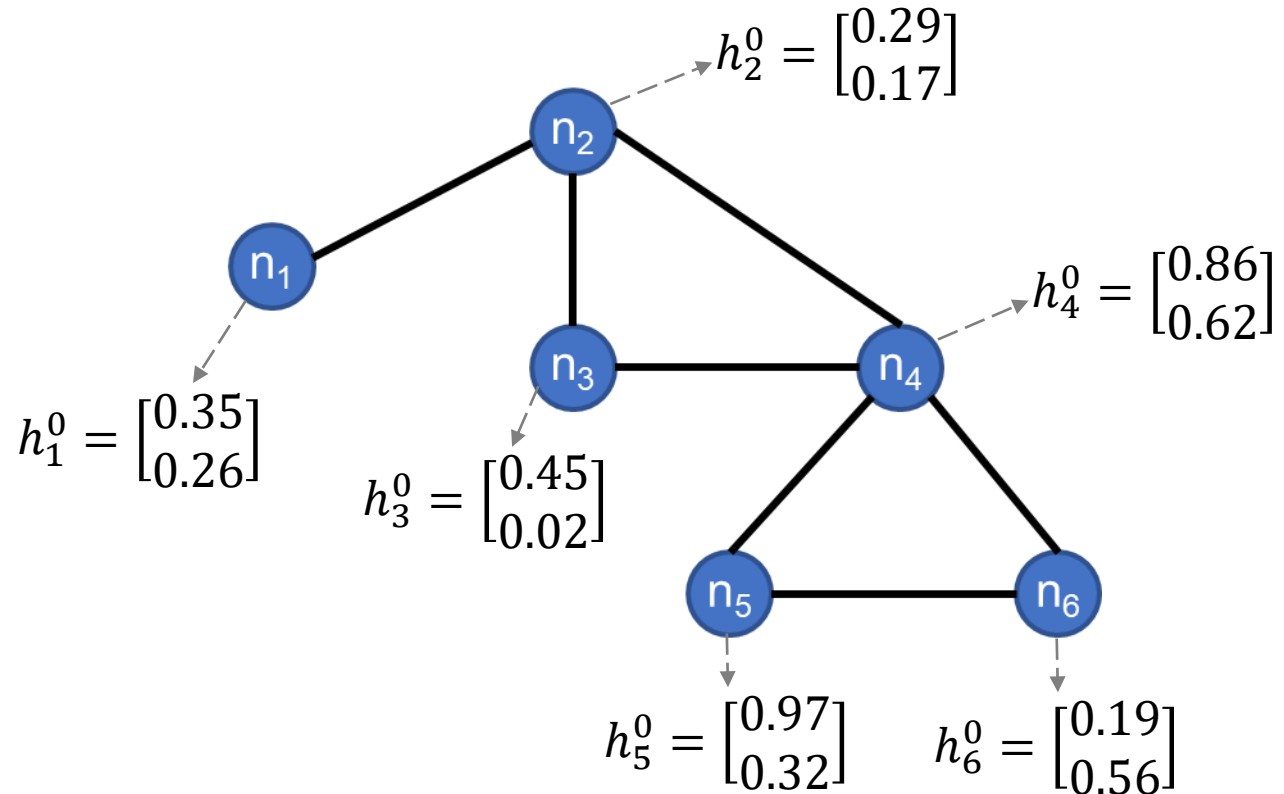
In the input layer, we represent each node/edge by its **features**, generating the vectors that are going to be **fed** to the GNN module.



What if we do not have features at all?

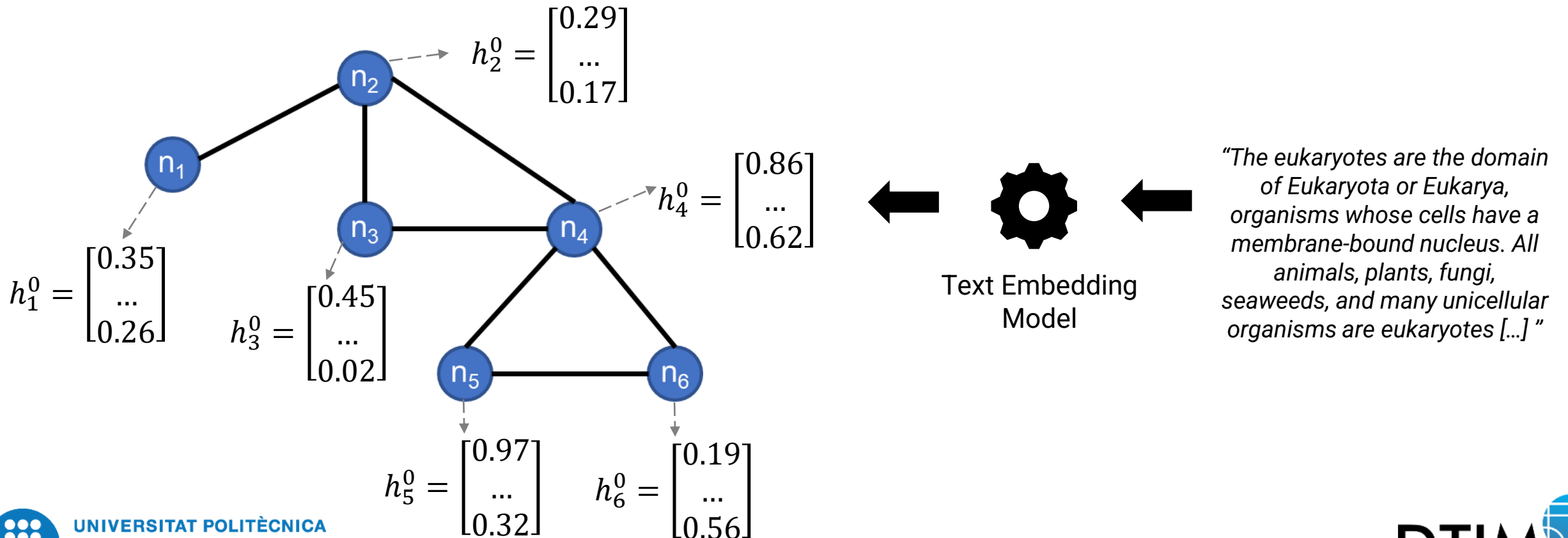
GNN: Input Layer

We can use **initialize** the initial vectors **randomly**, and let the model learn with the **graph structure**.

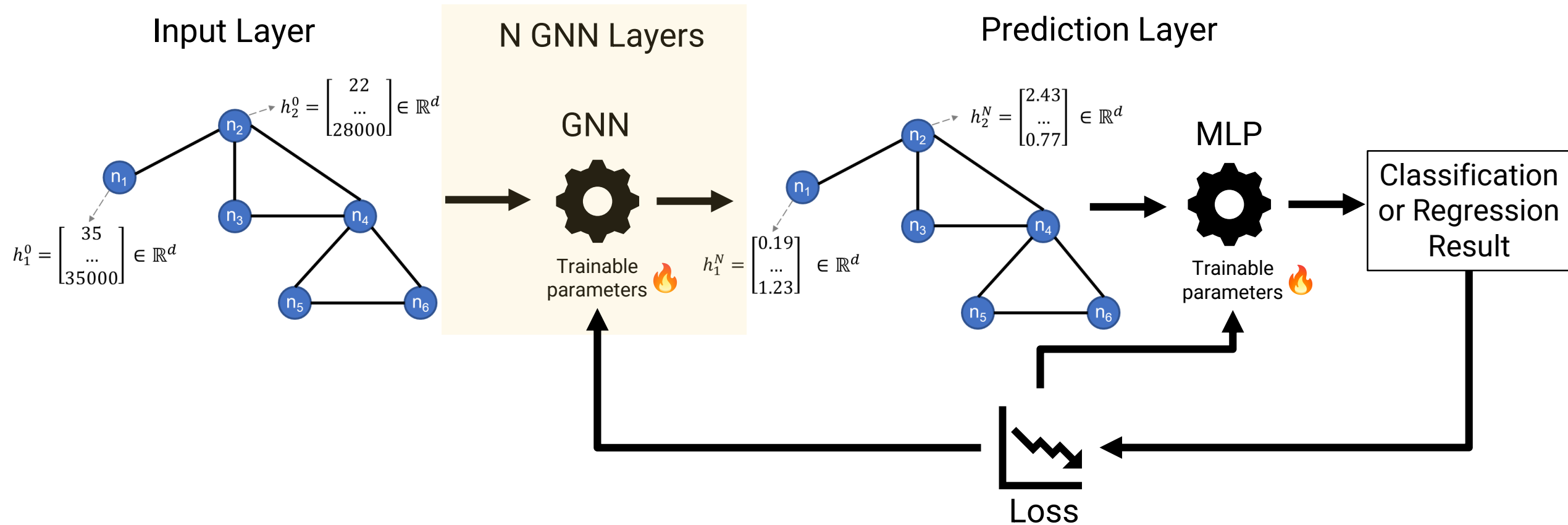


GNN: Input Layer

Another technique would be to use **textual information** associated with the node (e.g., node name, node description) and use **text embedding** models.

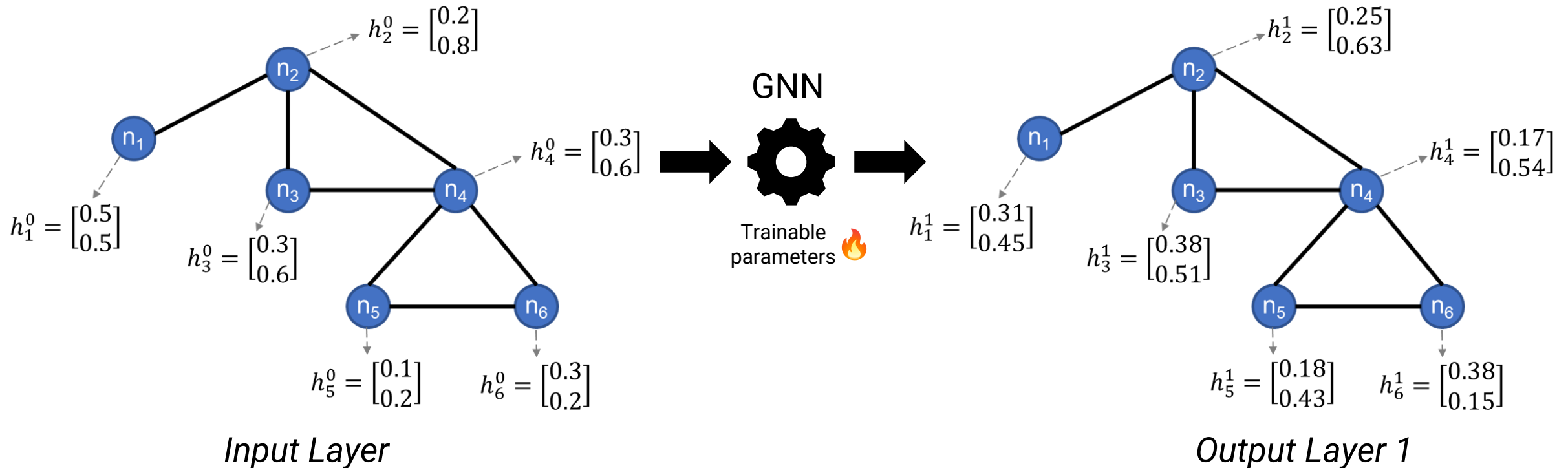


GNN: Basic Recipe



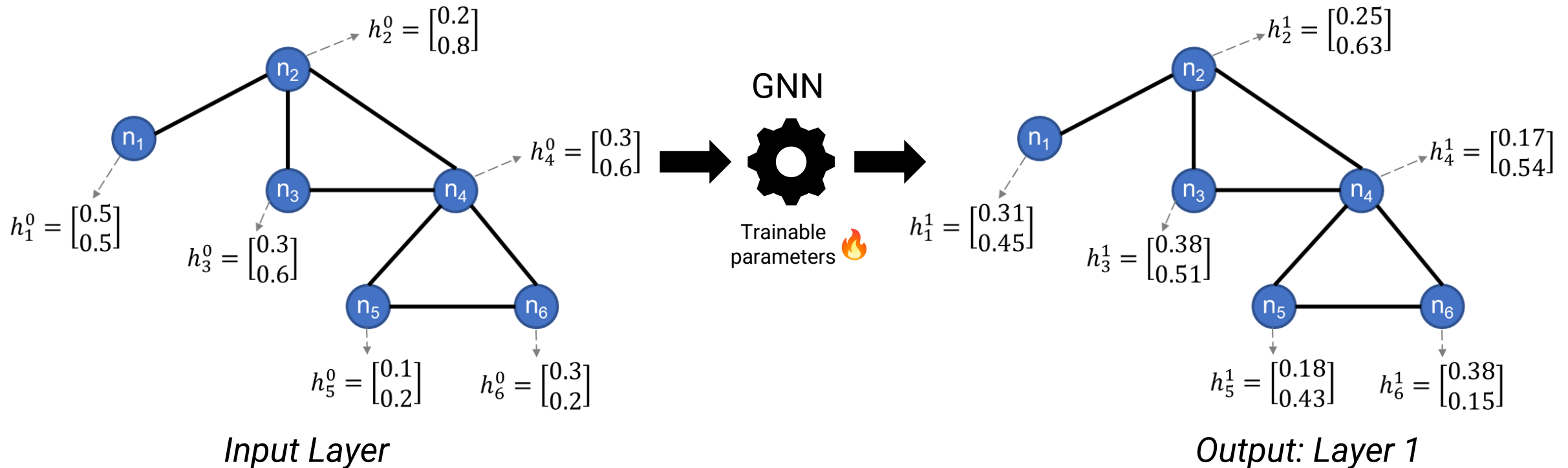
GNN: GNN Layers

The goal of GNNs is to transform the original representations into more **useful** ones, capturing both **graph structure** and **node/edge** information.



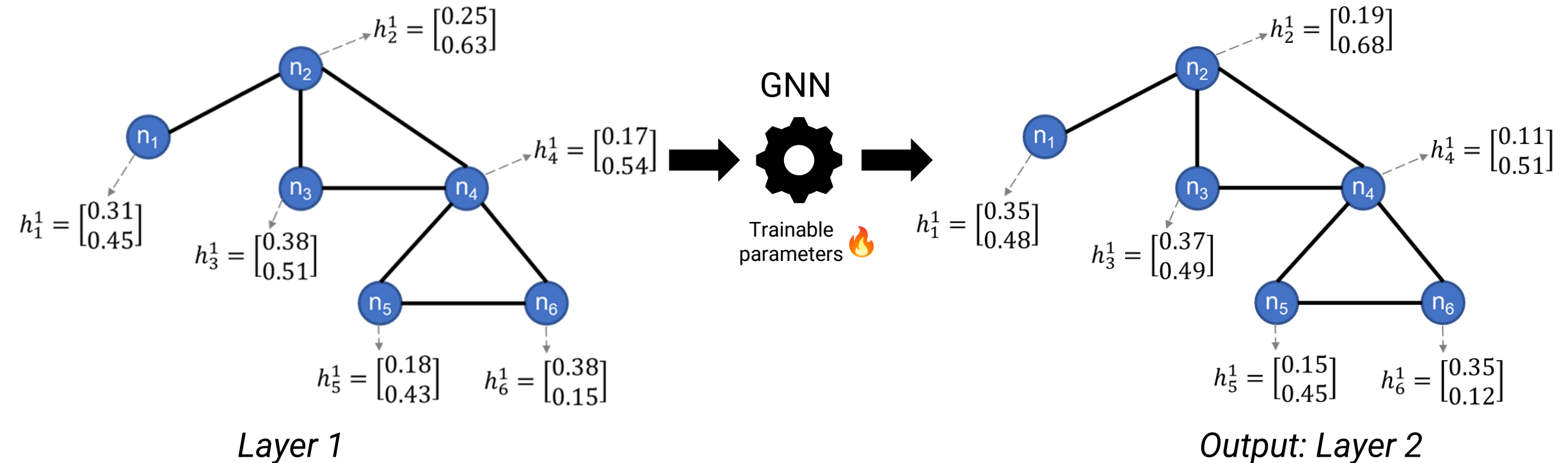
GNN: GNN Layers

GNNs are typically structured in **layers**, with the goal of letting each node incorporate information from progressively **farther neighborhoods** in the graph.



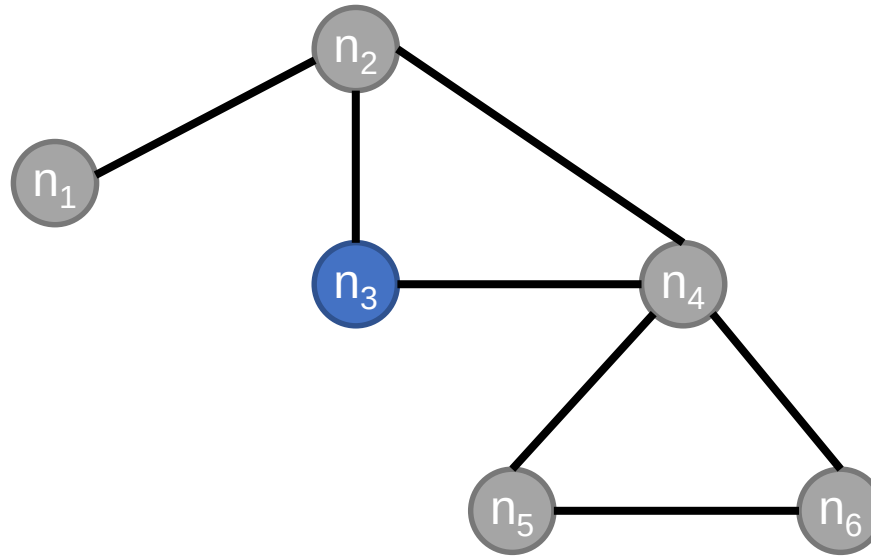
GNN: GNN Layers

GNNs are typically structured in **layers**, with the goal of letting each node incorporate information from progressively **farther neighborhoods** in the graph.



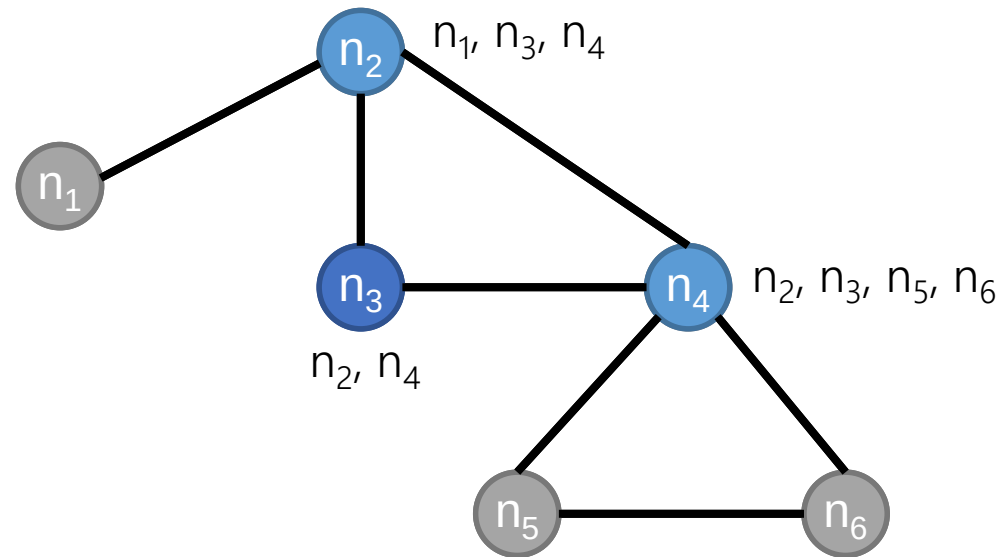
GNN: GNN Layers

GNNs are typically structured in **layers**, with the goal of letting each node incorporate information from progressively **farther neighborhoods** in the graph.



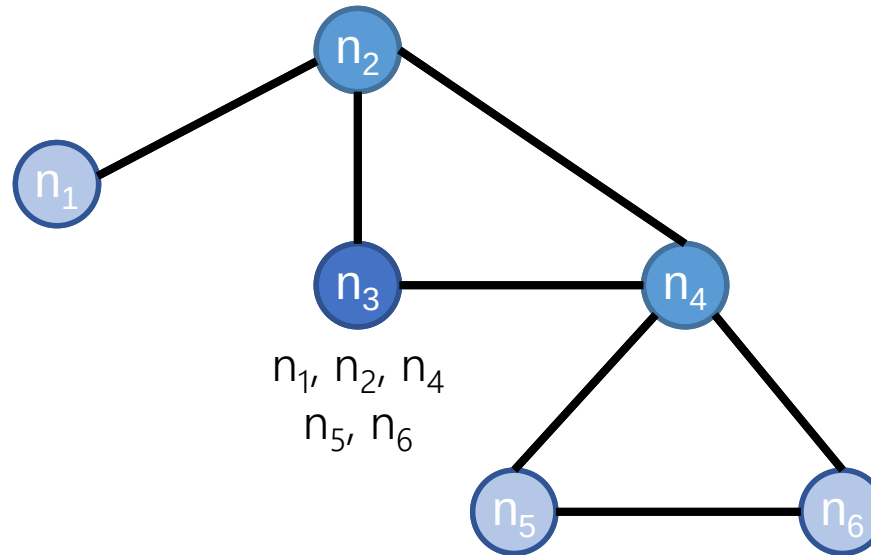
GNN: GNN Layers

GNNs are typically structured in **layers**, with the goal of letting each node incorporate information from progressively **farther neighborhoods** in the graph.



GNN: GNN Layers

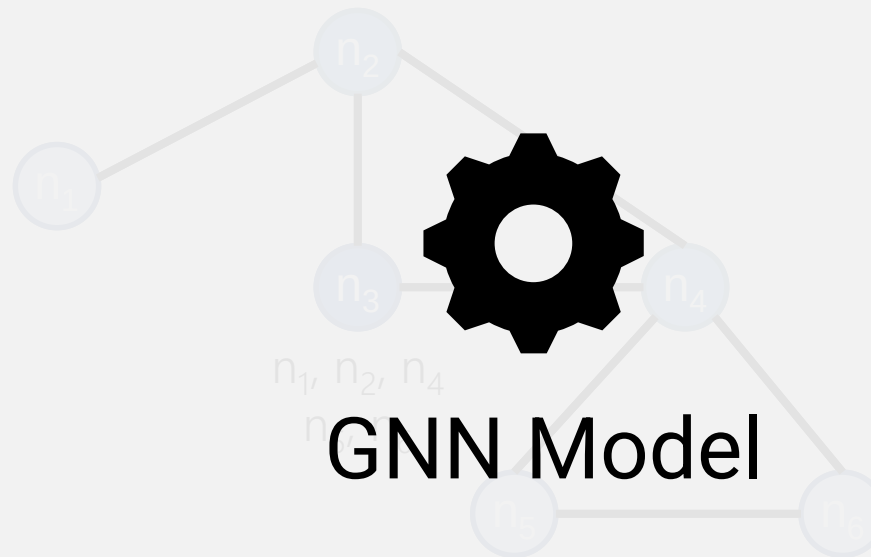
GNNs are typically structured in **layers**, with the goal of letting each node incorporate information from progressively **farther neighborhoods** in the graph.



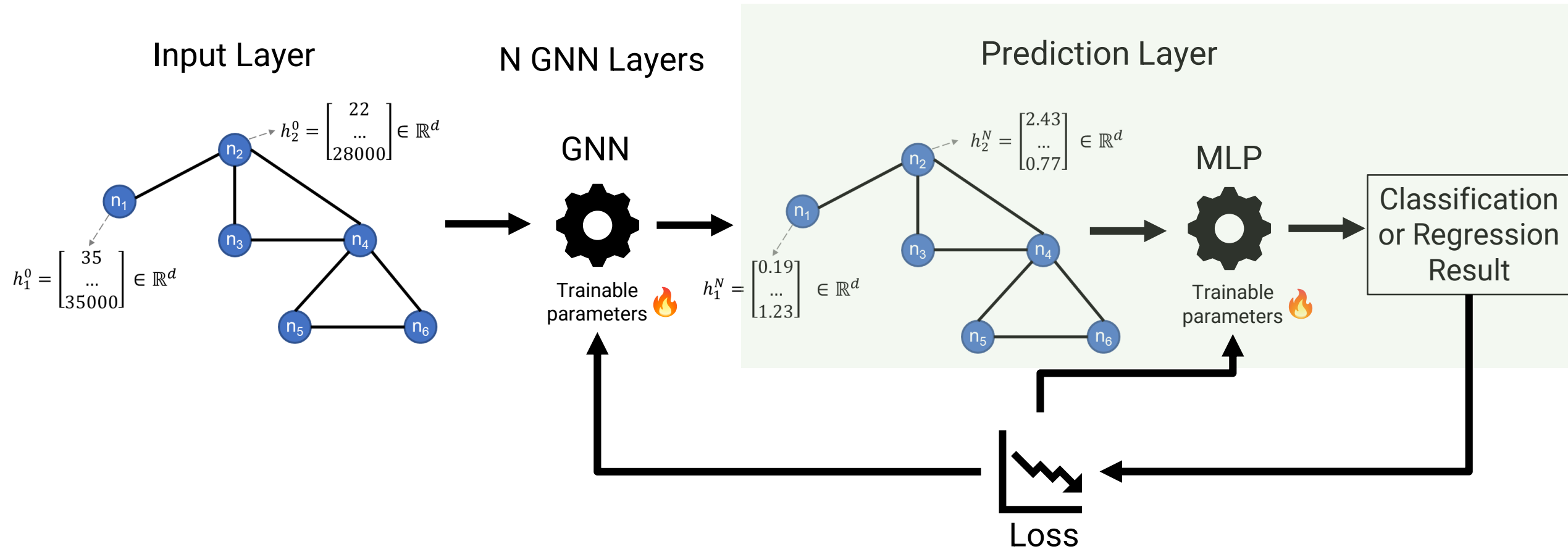
GNN: GNN Layers

GNNs are typically structured in **layers**, with the goal of letting each node incorporate information from progressively **farther neighborhoods** in the graph.

How do we propagate the information from one node to the other?

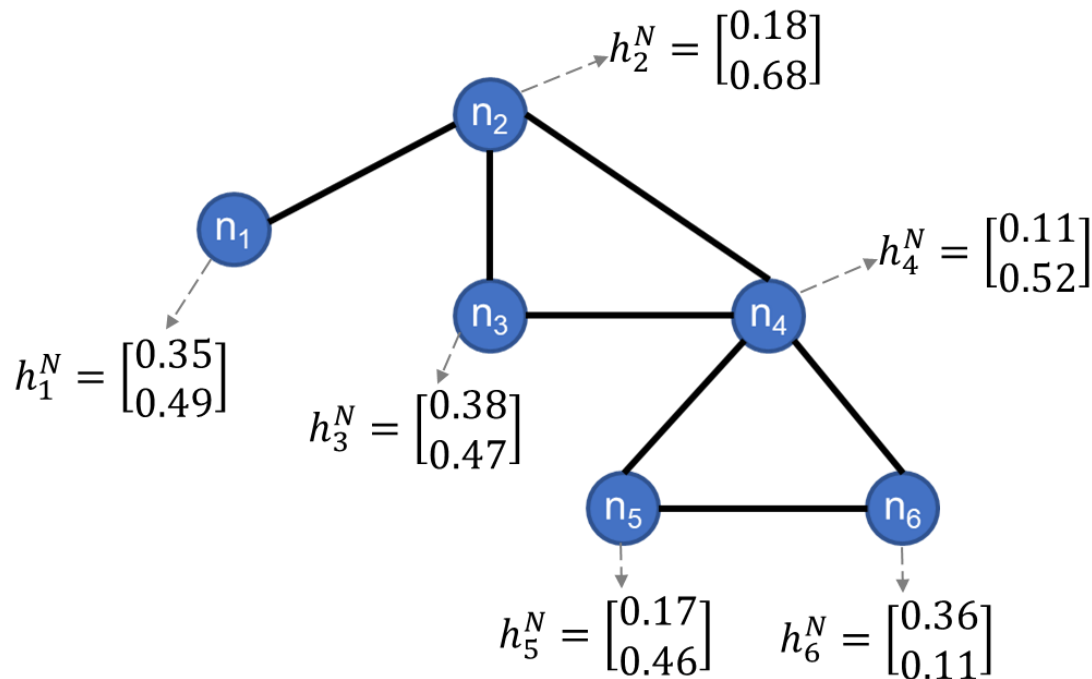


GNN: Basic Recipe



GNN: Prediction Layer (supervised GNNs)

Once we obtain the representations of the **last GNN layer**, we will feed them to a **prediction layer**, whose results will be used to **optimize** the **GNN parameters**.



Node Classification:

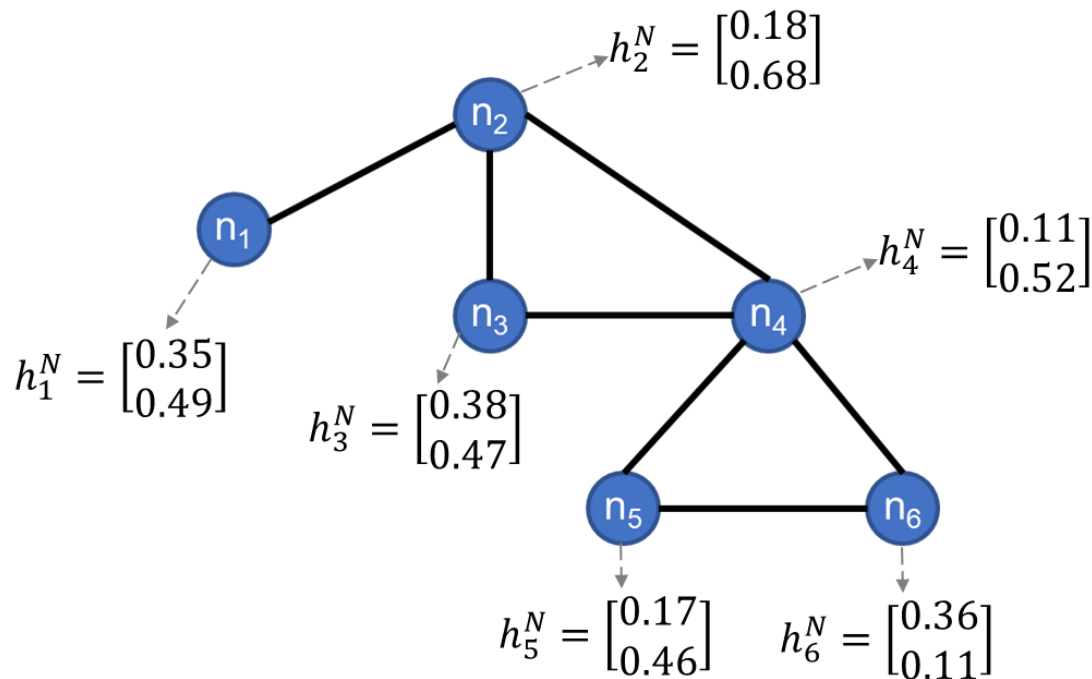
$$\hat{y}_i = \text{Softmax}(Wh_i^N)$$

$$\text{Loss}(y_i, \hat{y}_i)$$

We need to know y_i (i.e., the ground truth)

GNN: Prediction Layer (supervised GNNs)

Once we obtain the representations of the **last GNN layer**, we will feed them to a **prediction layer**, whose results will be used to **optimize** the **GNN parameters**.



Node Classification:

Imagine our graph represents the animal kingdom and we want to classify between carnivore, herbivore and omnivore:

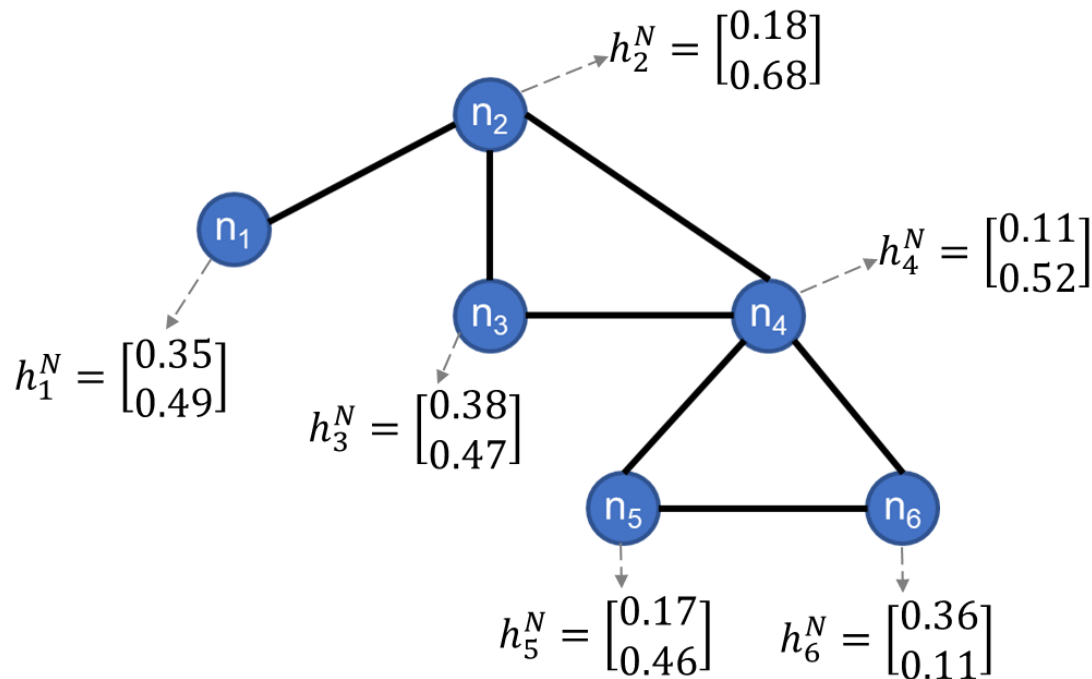
$$\hat{y}_1 = \text{Softmax}\left(\begin{pmatrix} 1.2 & 2.1 \\ 0.7 & 3.6 \\ 0.3 & 2.2 \end{pmatrix} \begin{bmatrix} 0.35 \\ 0.49 \end{bmatrix}\right)$$

$$\hat{y}_1 = \begin{pmatrix} 0.28 \\ 0.51 \\ 0.21 \end{pmatrix} \begin{matrix} p(\text{carnivore}) \\ p(\text{herbivore}) \\ p(\text{omnivore}) \end{matrix} \quad y_1 = \text{herbivore}$$

$$\text{Loss} = 0.67$$

GNN: Prediction Layer (supervised GNNs)

Once we obtain the representations of the **last GNN layer**, we will feed them to a **prediction layer**, whose results will be used to **optimize** the **GNN parameters**.



Graph Classification:

$$h^N = \frac{\sum_{i=1}^k h_i^N}{k}$$

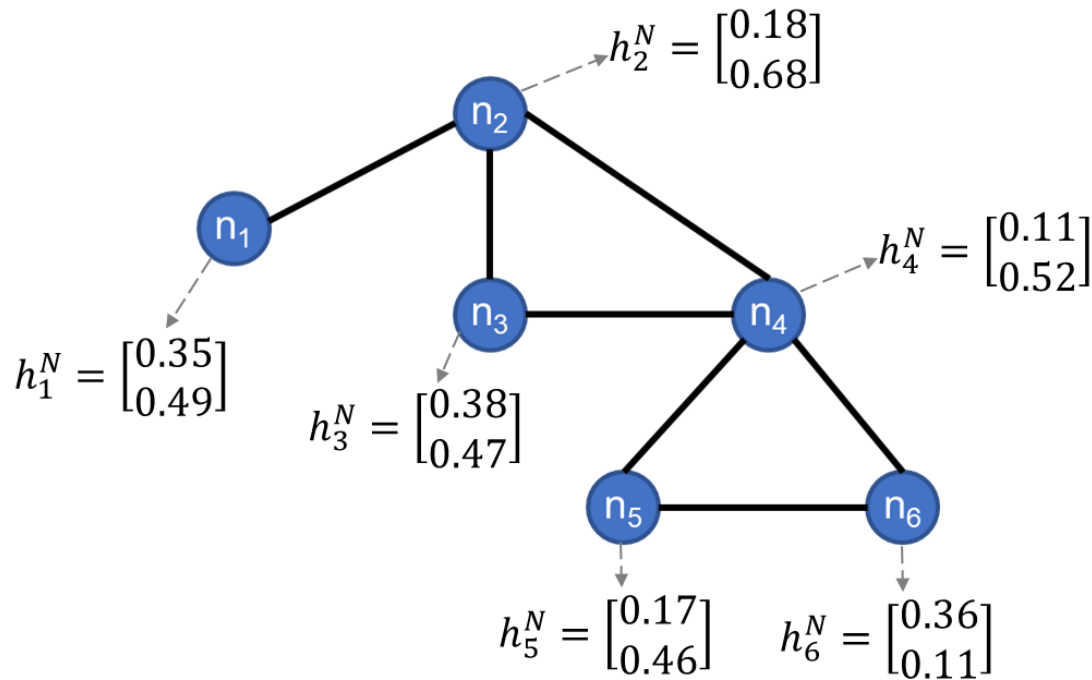
$$\hat{y} = \text{Softmax}(Wh^N)$$

$$\text{Loss}(y, \hat{y})$$

We need to know y (i.e., the ground truth)

GNN: Prediction Layer (self-supervised GNNs)

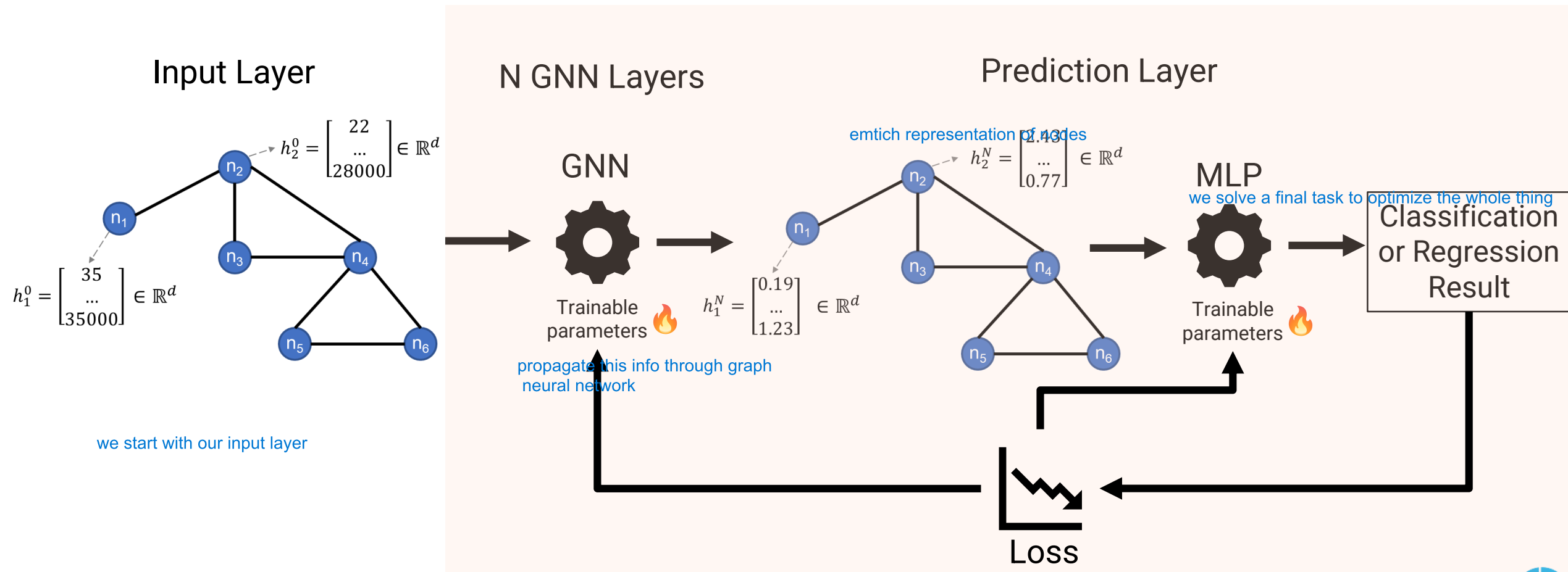
Once we obtain the representations of the **last GNN layer**, we will feed them to a **prediction layer**, whose results will be used to **optimize** the **GNN parameters**.



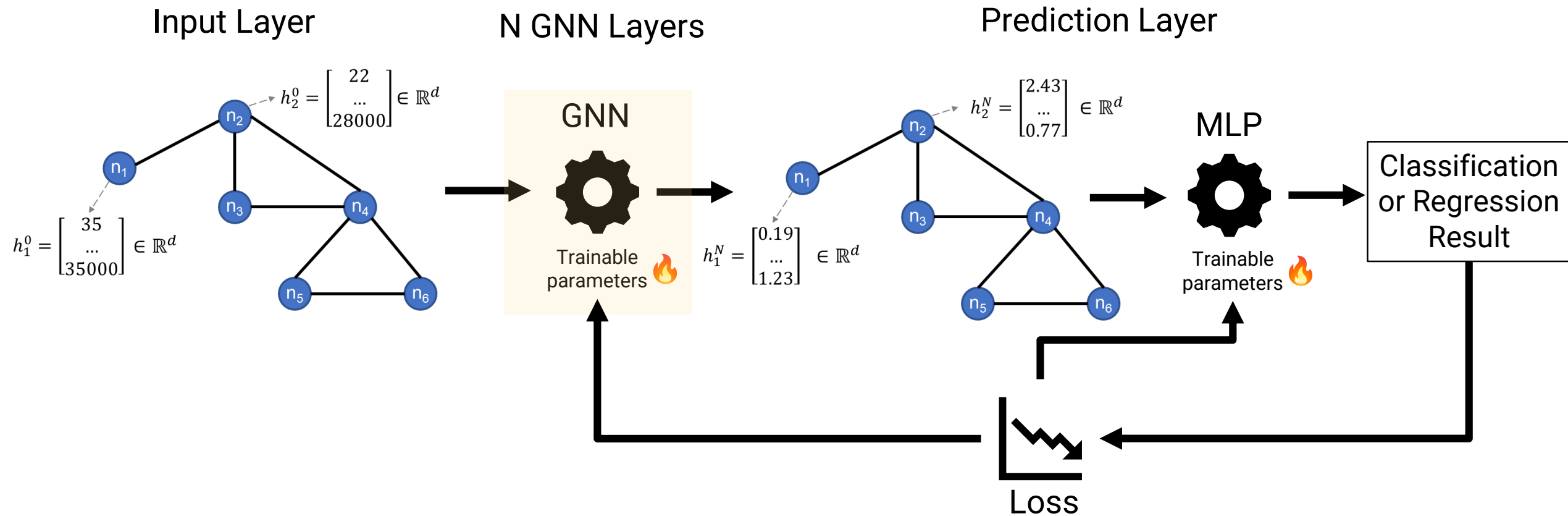
Edge Prediction:

Given two nodes, we predict if there exist an **edge** between them. This is the same principle as **Link Prediction** in KGE, thus this layer may or may not (e.g., multiplying the numbers) need **trainable parameters**.

GNN: Basic Recipe



GNN: Basic Recipe



GNN: **Message Passing**

The basic idea of **Message Passing** GNNs is to **iteratively** (i.e., layer by layer) update the node representations through the combination of their **own** representation and the representations of **its neighbors**. It is based on two important **functions**:

- **Aggregate**: defines how to aggregate the embeddings of the **neighbors** of a node.
- **Combine**: defines how to combine the aggregated information with its **current** representation.

GNN: Message Passing

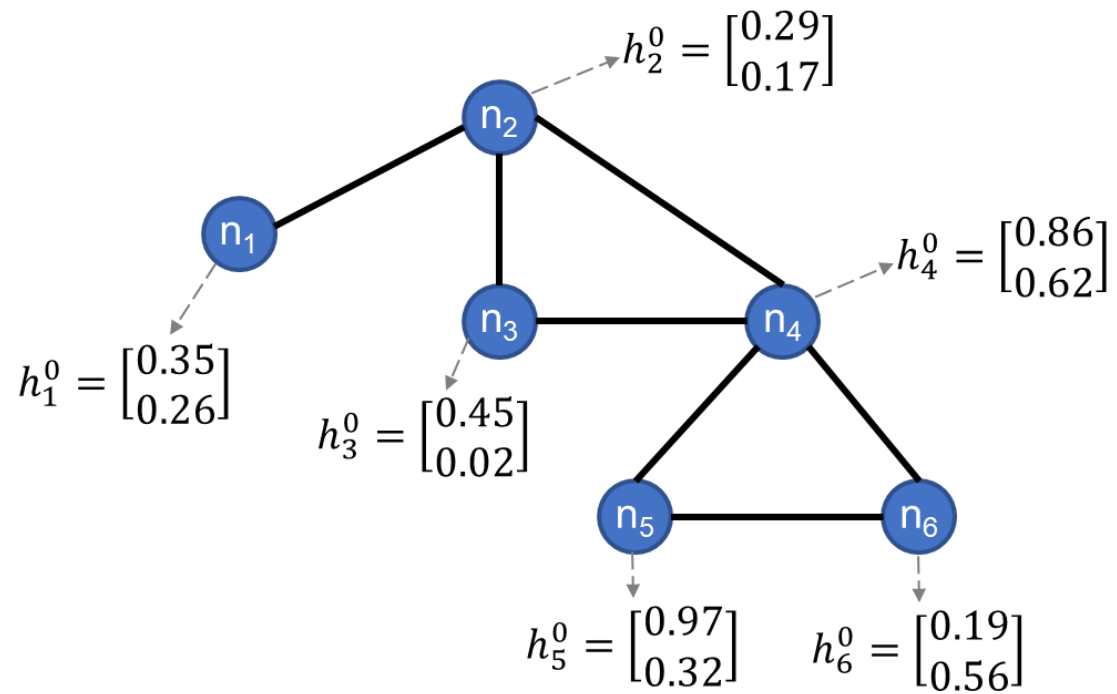
The **basic notation** we will use to understand GNNs is:

$h_i^L \longrightarrow$ **Representation** of node **i** at layer **L**.

$N(i) \longrightarrow$ **Neighborhood** of node **i**.

GNN: Message Passing

The **basic notation** we will use to understand GNNs is:



$$N(3) = \{2, 4\}$$

GNN: Message Passing

The **most simple** GNN **formulation** can be expressed like:

$$h_i^L = \sigma(W_{self}^L h_i^{L-1} + W_{neigh}^L \sum_{j \in N(i)} h_j^{L-1})$$

Combine

Aggregate

GNN: Message Passing

The **most simple** GNN **formulation** can be expressed like:

$$h_i^L = \sigma(\underbrace{W_{self}^L}_{\text{🔥}} h_i^{L-1} + \underbrace{W_{neigh}^L}_{\text{🔥}} \sum_{j \in N(i)} h_j^{L-1})$$

Different **learnable parameter matrices** are applied to neighbors and the node itself.

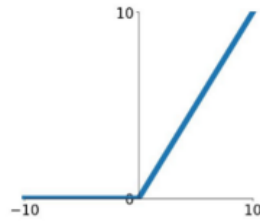
GNN: Message Passing

The **most simple** GNN **formulation** can be expressed like:

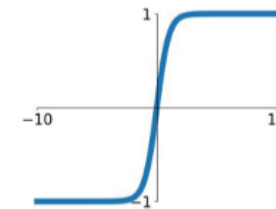
$$h_i^L = \sigma(W_{self}^L h_i^{L-1} + W_{neigh}^L \sum_{j \in N(i)} h_j^{L-1})$$

Different **activation functions** (e.g., ReLU, tanh...) are used to introduce non-linearities.

ReLU
 $\max(0, x)$

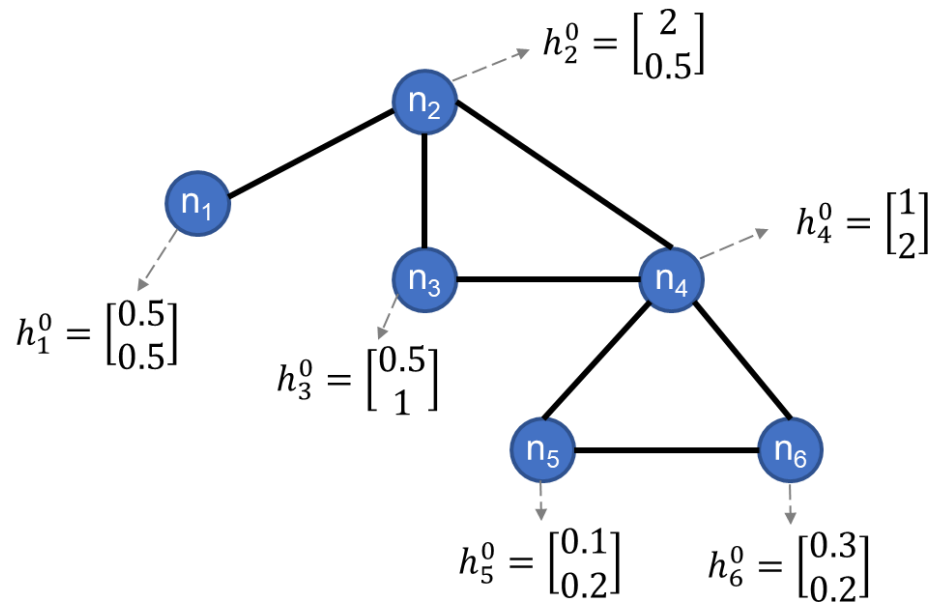


tanh
 $\tanh(x)$



GNN: Message Passing Exercise (5 minutes)

Obtain h_3^1



$$W_{self}^1 = \begin{bmatrix} 1 & 2 \\ 2 & 2 \end{bmatrix} \quad W_{neigh}^1 = \begin{bmatrix} 1 & -4 \\ 2 & 1 \end{bmatrix} \quad \sigma() := ReLU()$$

GNN: Message Passing

One decision in the design of GNNs is **neighbor normalization**, as nodes with high degree obtain large scale updated representations, which can **explode** when applying multiple layers and also disturb the **influence** of these nodes in future layers.

$$h_i^L = \sigma(W_{self}^L h_i^{L-1} + W_{neigh}^L \sum_{j \in N(i)} h_j^{L-1})$$

GNN: Message Passing

One decision in the design of GNNs is **neighbor normalization**, as nodes with high degree obtain large scale updated representations, which can **explode** when applying multiple layers and also disturb the **influence** of these nodes in future layers.

$$h_i^L = \sigma(W_{self}^L h_i^{L-1} + W_{neigh}^L \frac{1}{|N(i)|} \sum_{j \in N(i)} h_j^{L-1})$$

Cons: This can lead to **loss of structural information** (e.g., effect of nodes with different degree) or **over-smoothing** (i.e., node representations become increasingly similar, especially in deep GNNS).

GNN: **Message Passing**

We are now going to see three common **GNN architectures**:

- Graph Convolutional Networks (**GCN**)
- **GraphSAGE**
- Graph Attention Networks (**GAT**)

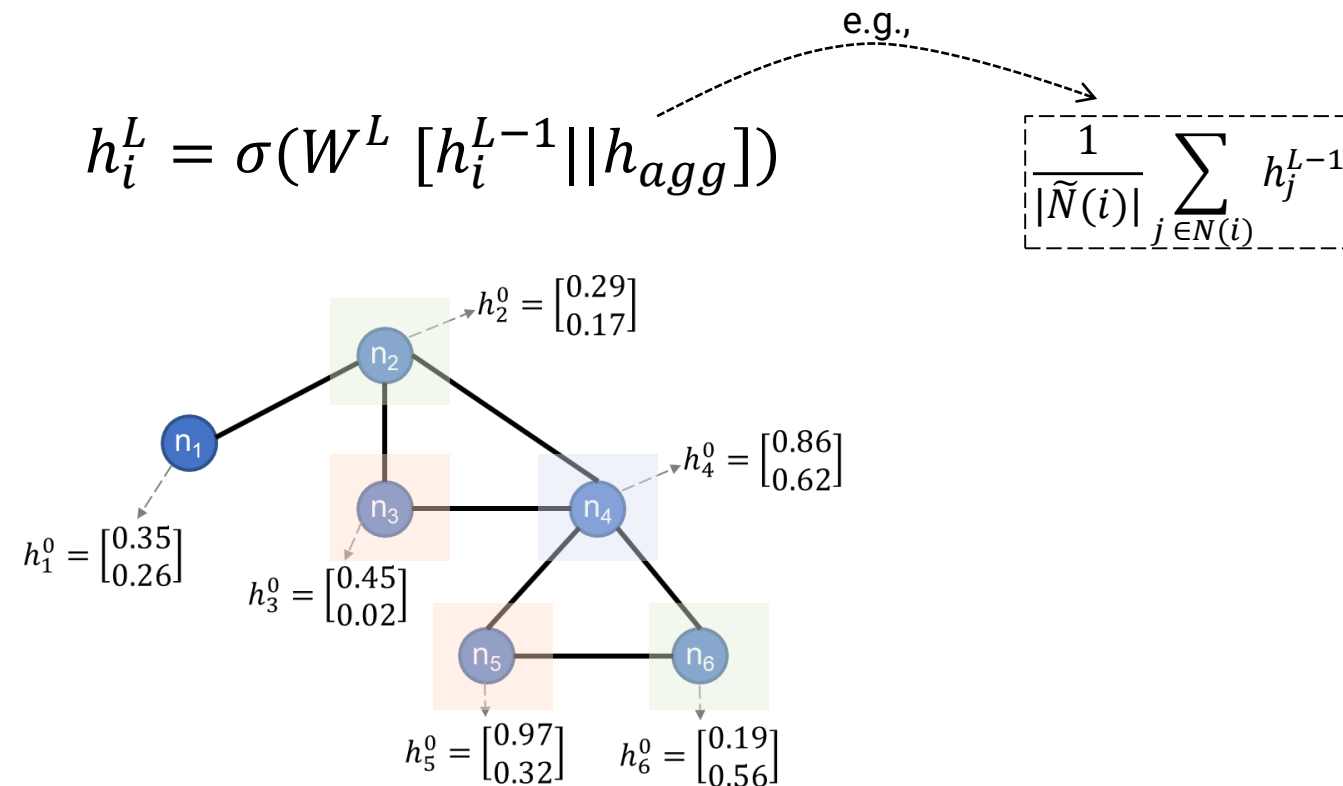
GNNs: GCN

This is one of the most **simple** approaches, which does not differentiate between the neighbors and the node itself, and **normalizes** the representations depending on the degree of the target node and the one of its neighbors:

$$h_i^L = \sigma(W^L \sum_{j \in N(i) \cup i} \frac{h_j^{L-1}}{\sqrt{|N(i)||N(j)|}})$$

GNNs: GraphSAGE

This approach introduces **sampled neighborhoods** in order to provide better scalability, **concatenation** of the vectors in the combine step, and **different aggregation** methods.



GNNs: GraphSAGE

This approach introduces **sampled neighborhoods** in order to provide better scalability, **concatenation** of the vectors in the combine step, and **different aggregation** methods.

$$h_i^L = \sigma(W^L [h_i^{L-1} || h_{agg}])$$

$\mathbb{R}^d$ $\mathbb{R}^{d \times 2d}$ $\mathbb{R}^d$ $\mathbb{R}^d$

$\mathbb{R}^{2d}$

GNNs: GraphSAGE

This approach introduces **sampled neighborhoods** in order to provide better scalability, **concatenation** of the vectors in the combine step, and **different aggregation** methods.

$$h_i^L = \sigma(W^L [h_i^{L-1} || h_{agg}])$$

Why does it need to be a **simple sum**?

$$\frac{1}{|N(i)|} \sum_{j \in N(i)} h_j^{L-1}$$

Max Pooling

$$\max_{j \in N(i)}^{element} (W_{pool}^L h_j^{L-1})$$

$$\begin{bmatrix} 2 \\ 3 \end{bmatrix} \begin{bmatrix} 4 \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

Arbitrarily Complex NN (e.g., LSTM)

$$NN_1 \left(\sum_{j \in N(i)} NN_2(h_j^{L-1}) \right)$$

GNNs: GAT

This GNNs are based on **attention**. This mechanism was first introduced for text processing:

The chef prepared the soup in the kitchen.

*Do all the words **contribute equally** to its meaning?*

Different weights are computed for each word pair, to capture different relevant aspects (e.g., one weight could be useful to capture the **subject-verb interaction**, assigning a high value to the word *chef*, while others could focus on **direct object**, highlighting the word *soup*).

GNNs: GAT

This GNNs are based on **attention**. This means that the all nodes do **not contribute equally** to the aggregation. Various **attention blocks** can be defined to give importance to different aspects.

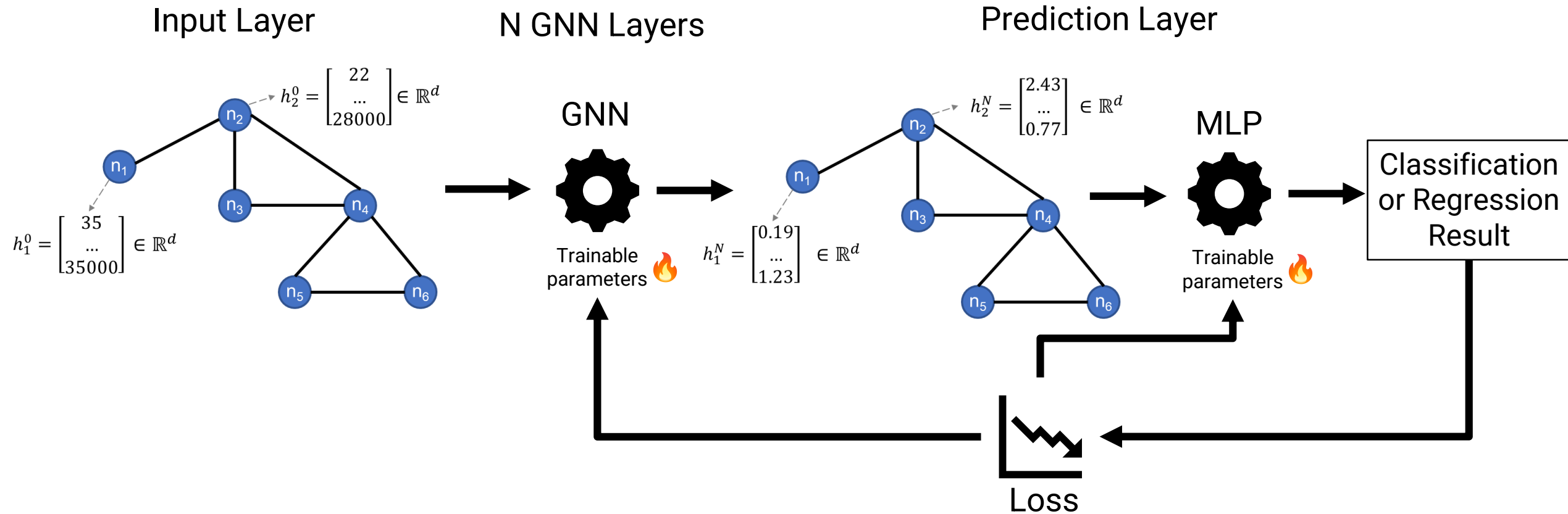
$$\alpha_{i,j}^k = \frac{\exp(h_i^{L-1} U^k h_j^{L-1})}{\sum_{m \in N(i)} h_i^{L-1} U^k h_m^{L-1}}$$

Importance that target **node i** should give to its **neighbor j**.

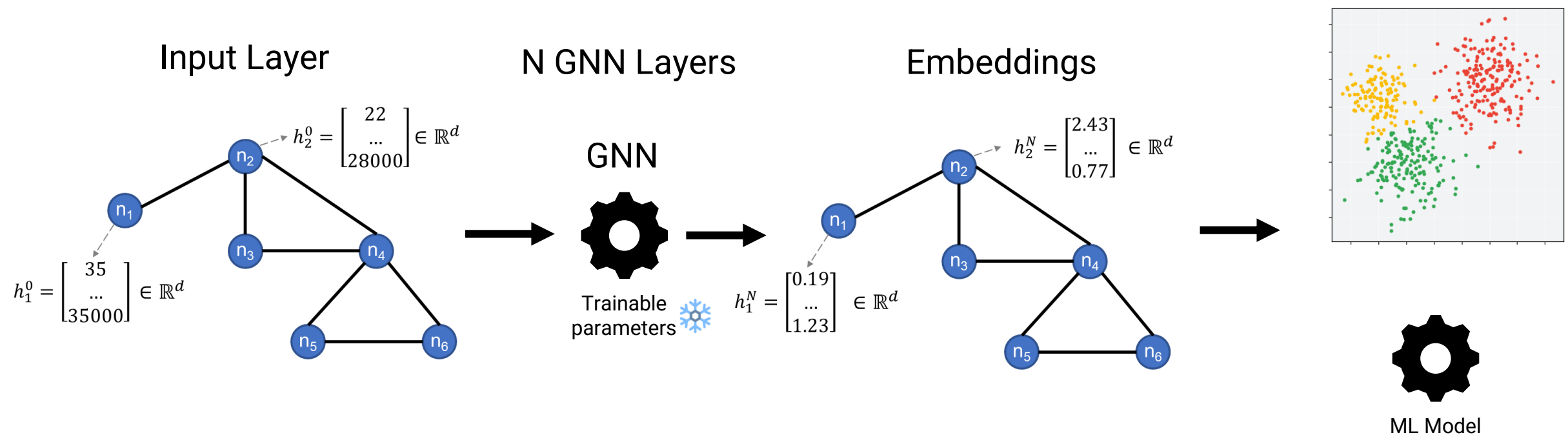
$$h_i^L = \sigma\left(\frac{1}{K} \sum_{k=1}^K \sum_{j \in N(i)} \alpha_{i,j}^k W^k h_j^{L-1}\right)$$

We can perform **average aggregation**, **concatenation**, etc.

GNN: Embeddings



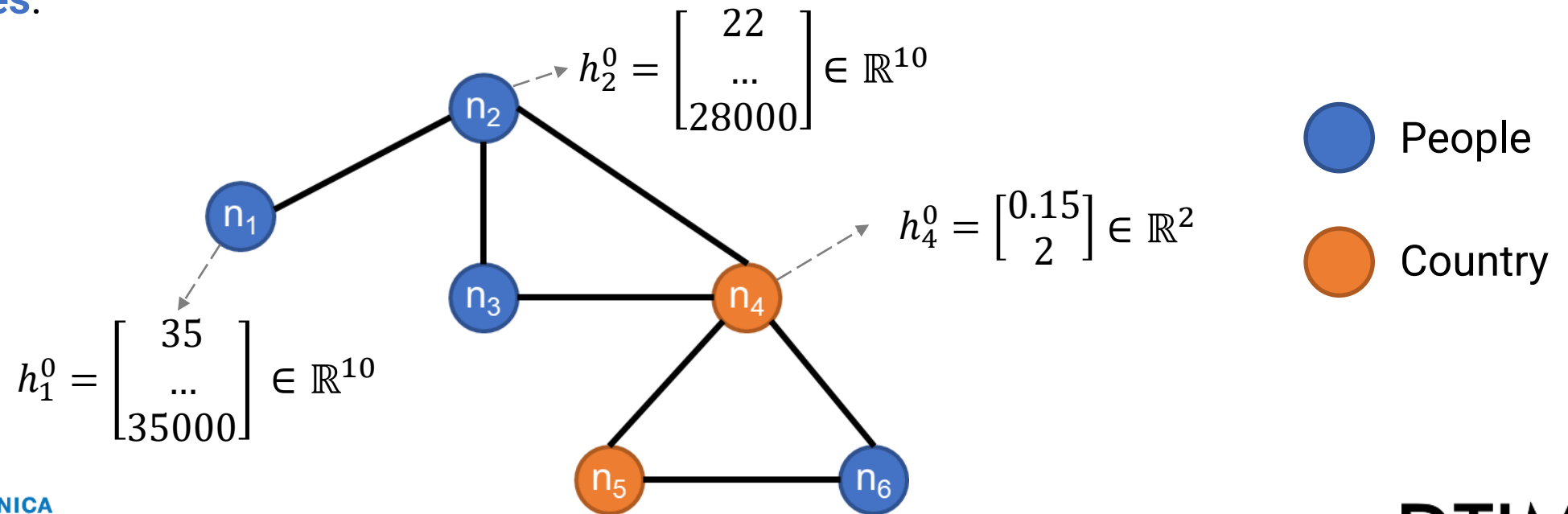
GNN: Embeddings



GNN: How well do Knowledge and Property Graphs fit in GNNs?

By now, we have seen GNNs focusing on **simple graph** and single node representations. However, to accommodate Property and Knowledge Graph, we need slightly **more complex** architectures:

- **Nodes** (i.e., entities) may have different classes, which may result in **different available properties**.



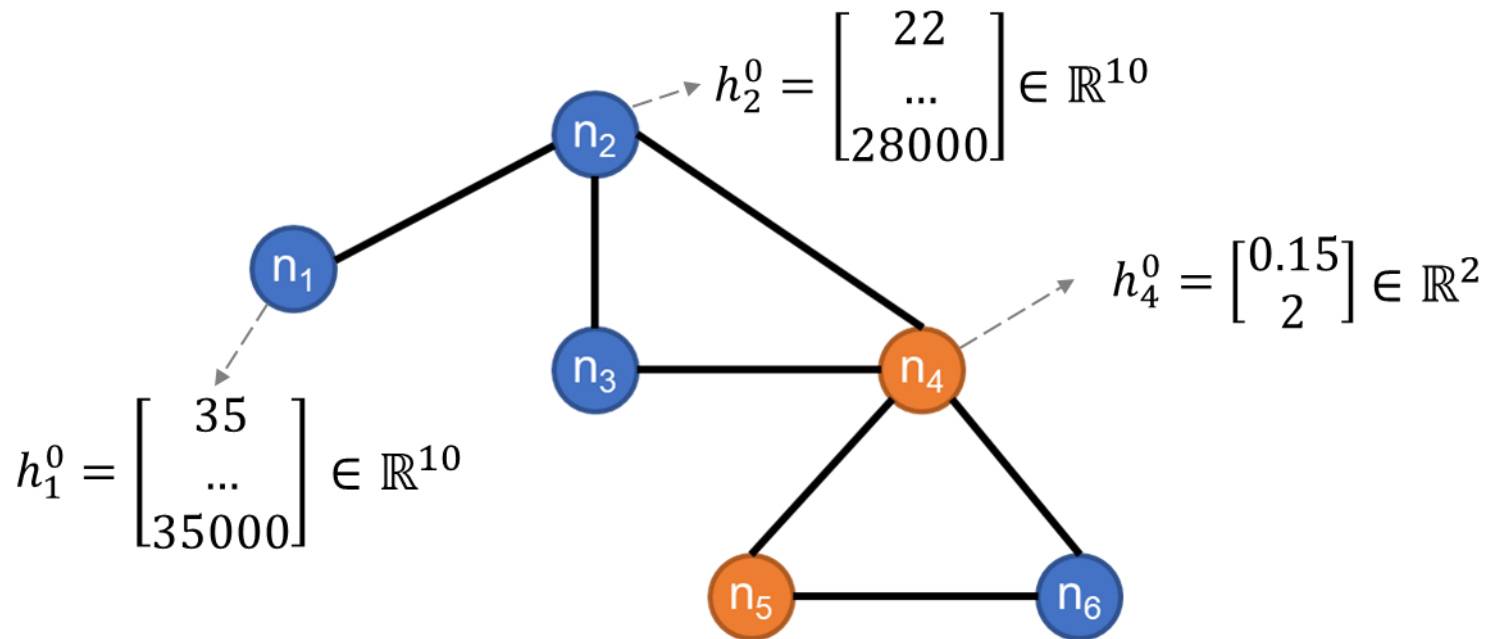
GNN: How well do Knowledge and Property Graphs fit in GNNs?

By now, we have seen GNNs focusing on **simple graph** and single node representations. However, to accommodate Property and Knowledge Graph, we need slightly **more complex** architectures:

- **Nodes** (i.e., entities) may have different classes, which may result in **different available properties**.
- Different types of **edges** (i.e., relations) exist, which should **contribute differently** to the **aggregation**.

GNN: Nodes with Different Properties

If the input nodes have features of **different dimensionality**, we can use **different linear projections** for each one, obtaining vector of the same dimension.



$$h_i'^0 \in \mathbb{R}^{100}$$

$$h_{\blacksquare}^0 \in \mathbb{R}^{10} \quad h_{\blacksquare}^0 \in \mathbb{R}^2$$

$$W_{\blacksquare} \in \mathbb{R}^{100 \times 10}$$

$$W_{\blacksquare} \in \mathbb{R}^{100 \times 2}$$

$$h_{\blacksquare}^{'0} = W_{\blacksquare} h_{\blacksquare}^0 \quad h_{\blacksquare}^{'0} = W_{\blacksquare} h_{\blacksquare}^0$$

GNN: Different Relations

To take **different relations** into account, some modifications can be done in the GNN architecture, as in Relational Graph Convolutional Networks (**RGCN**).

$$h_i^L = \sigma(W_{self}^L h_i^{L-1} + W_{neigh}^L \sum_{j \in N(i)} h_j^{L-1}) \quad \text{Non-Relational}$$

$$h_i^L = \sigma(W_{self}^L h_i^{L-1} + \sum_{r \in R} \sum_{j \in N_r(i)} W_{r,neigh}^L h_j^{L-1}) \quad \text{Relational}$$

GNN: Different Relations

To take **different relations** into account, some modifications can be done in the GNN architecture, as in Relational Graph Convolutional Networks (**RGCN**).

$$h_i^L = \sigma(W_{self}^L h_i^{L-1} + \sum_{r \in R} \sum_{j \in N_r(i)} W_{r,neigh}^L h_j^{L-1})$$

This means that we have $|R|$ times more parameters!

GNN: Different Relations

To take **different relations** into account, some modifications can be done in the GNN architecture, as in Relational Graph Convolutional Networks (**RGCN**).

$$h_i^L = \sigma(W_{self}^L h_i^{L-1} + \sum_{r \in R} \sum_{j \in N_r(i)} W_{r,neigh}^L h_j^{L-1})$$

To mitigate that, the weight matrices are defined by a **linear combination** of shared “**basis**” **matrices**:

$$W_r^L = \sum_{k=1}^b \alpha_{k,r} B_k$$

GNN: Different Relations

To mitigate that, the weight matrices are defined by a **linear combination** of shared “**basis**” **matrices**:

$$W_r = \sum_{k=1}^b \alpha_{k,r} B_k$$

$$W_1 = \alpha_{1,1} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + \alpha_{2,1} \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

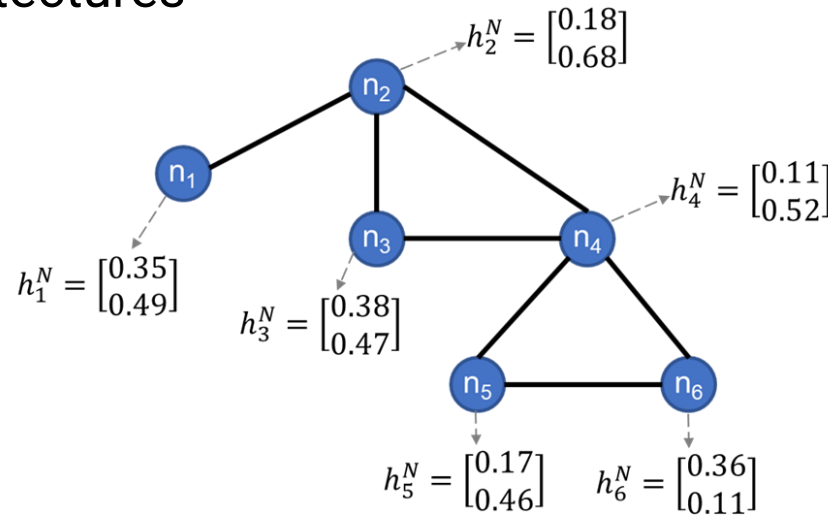
$$W_n = \alpha_{1,n} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + \alpha_{2,n} \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

So, given $|R|$ relations and a weight matrix of T parameters, we previously had $|R|*T$ total parameters, while now we have $b*(|R|+T)$.

GNN: How well do Knowledge and Property Graphs fit in GNNs?

However, there is still **room for improvement** to fully leverage these graphs for GNNs.

- Negative Sampling
- Loss Penalization
- Schema-Aware architectures
- ...

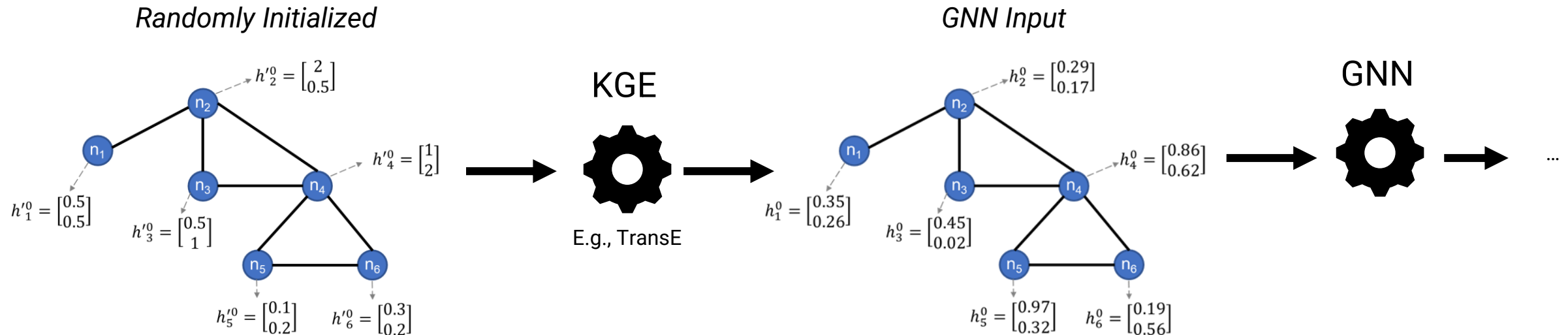


Edge Prediction:

Given two nodes, we predict if there exist an **edge** between them. This is the same principle as **Link Prediction** in KGE, thus this layer may or may not (e.g., multiplying the numbers) need **trainable parameters**.

GNN and KGE: Benefiting from Each Other

When exploring the **Input Layer**, we saw that if there properties from nodes do not exist, one solution is to **randomly initialize** the vectors. To mitigate this issue, it is common to **initialize** GNNs with the **resulting embeddings** of a “light-weight” KGE model (e.g., TransE).



GNN: Applications

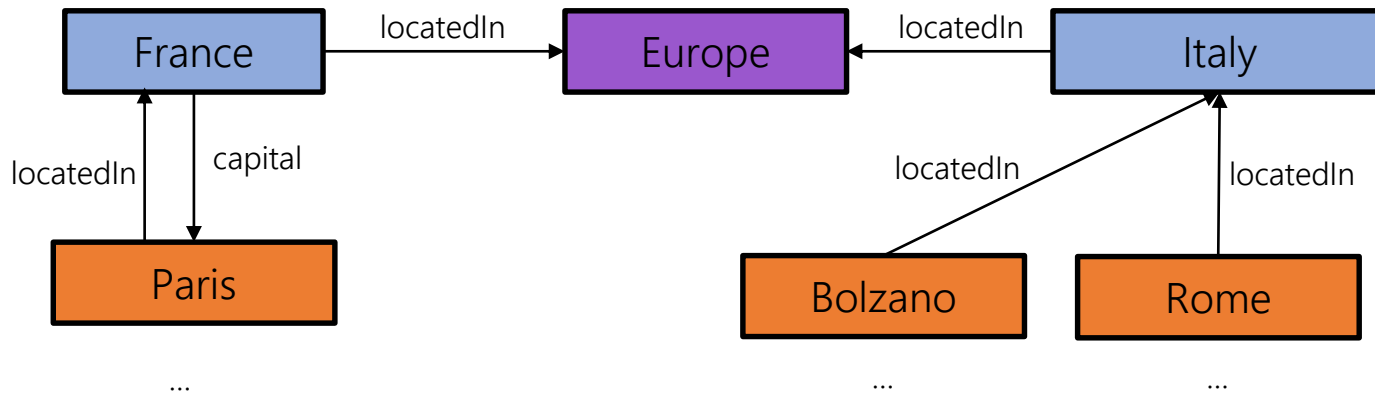
The techniques we have seen in this lecture can be used for:

- **Recommendation Systems:** as in KGE, we can perform Link Prediction.
- **Content Moderation:** by exploring how messages propagate over the network, fake news can be detected.
- **Molecular Property Prediction:** assess the molecular properties without slow and expensive lab experiments.
- **Traffic Forecasting:** model the spatial and temporal information in the graph.

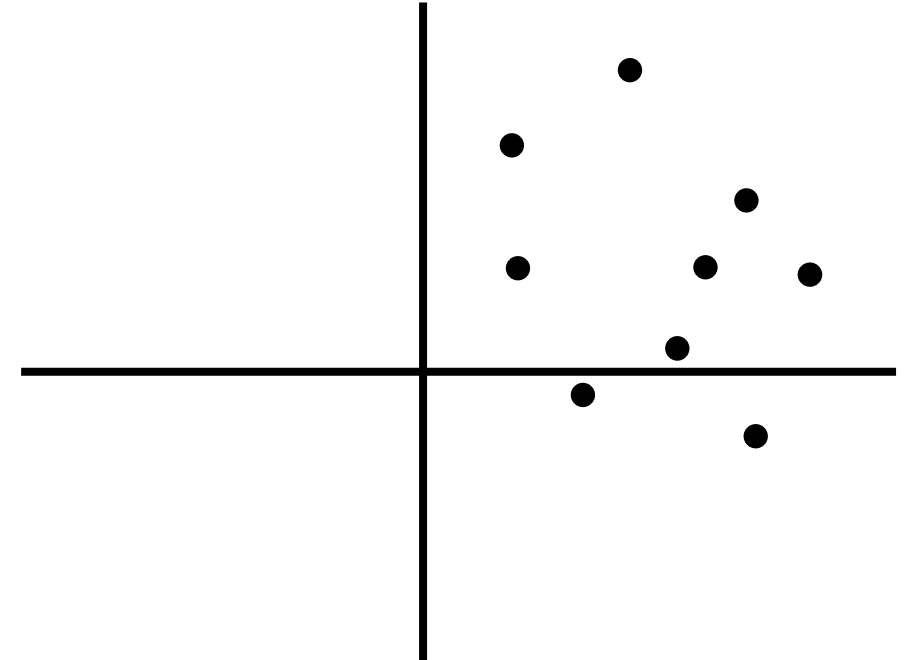
Link Prediction: Direct Applications

Generating embeddings allows to use well-know **machine learning** models over KG (e.g., classification, clustering,...). However, we can also use them directly:

Knowledge Graph Completion



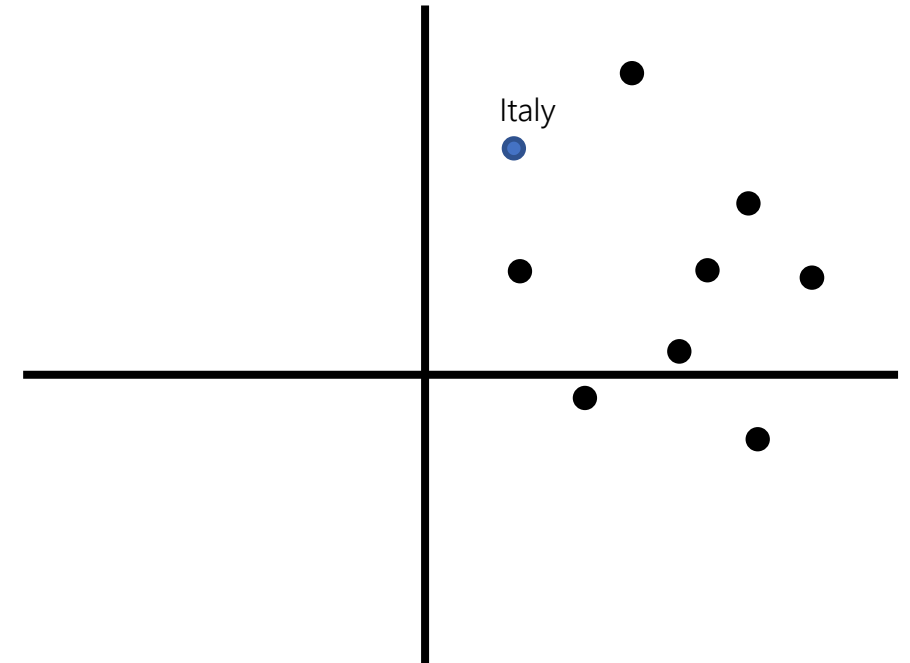
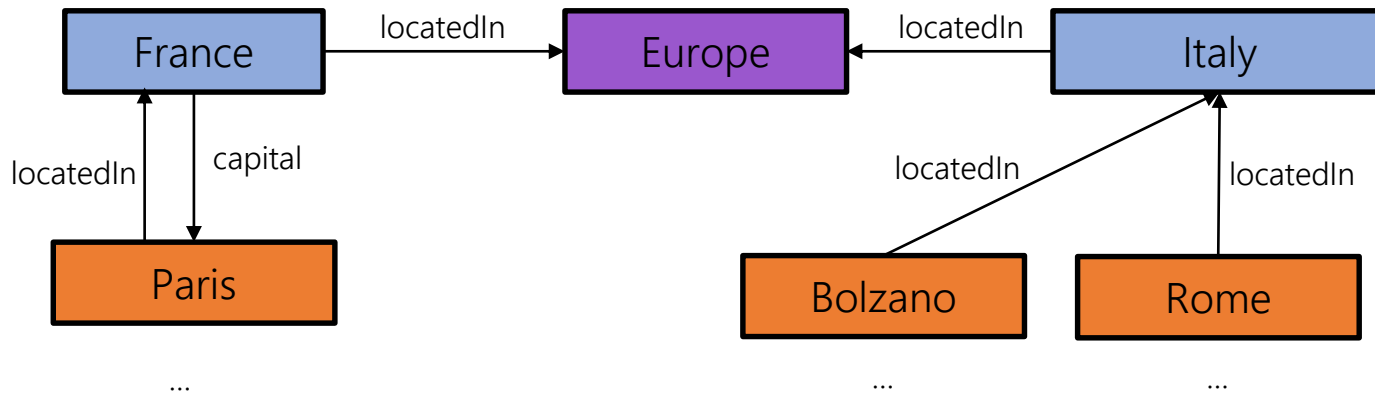
Visualization with KGEs, but the same can be done with GNNs!



Link Prediction: Direct Applications

Generating embeddings allows to use well-know **machine learning** models over KG (e.g., classification, clustering,...). However, we can also use them directly:

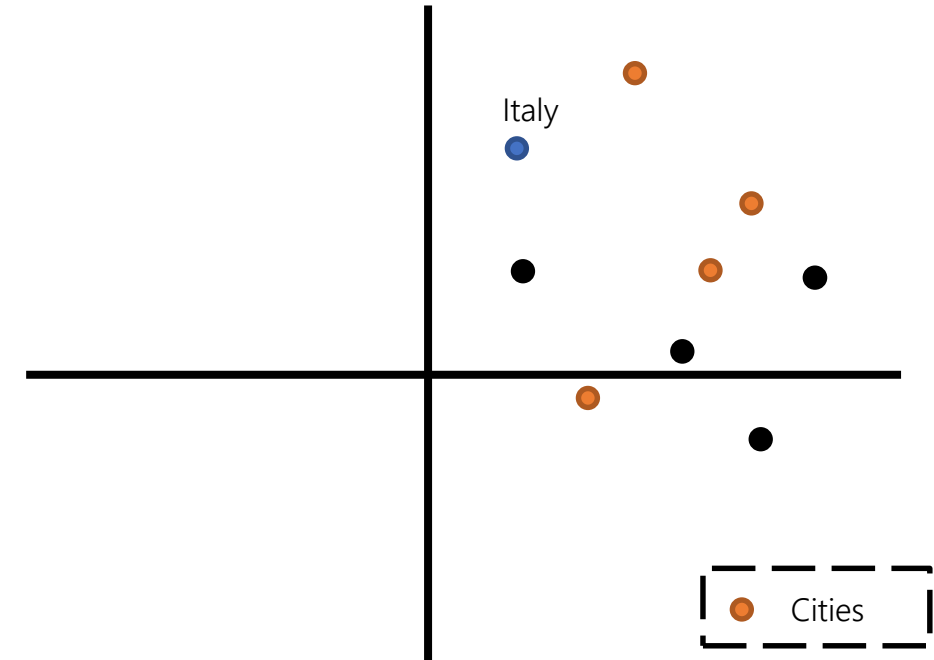
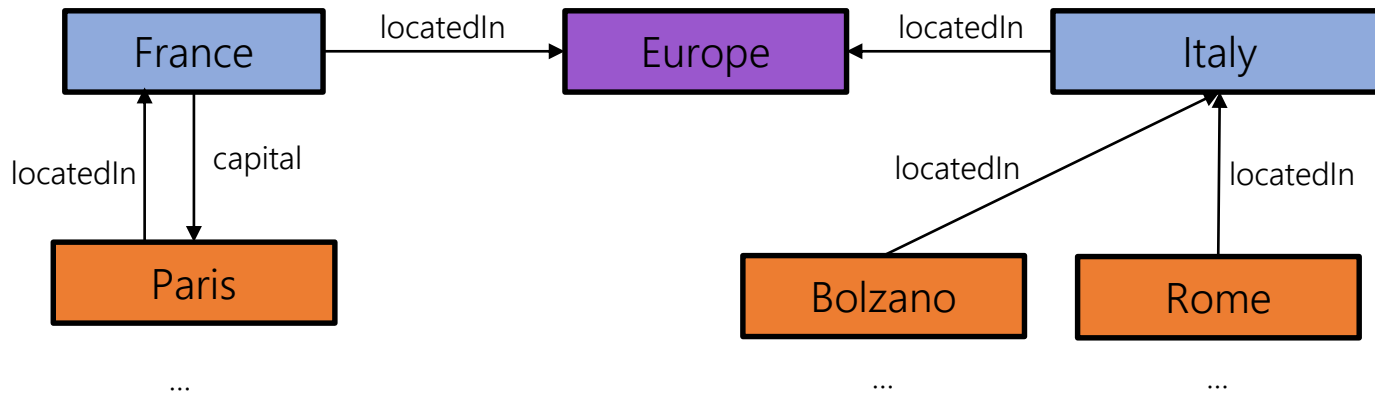
Knowledge Graph Completion



Link Prediction: Direct Applications

Generating embeddings allows to use well-know **machine learning** models over KG (e.g., classification, clustering,...). However, we can also use them directly:

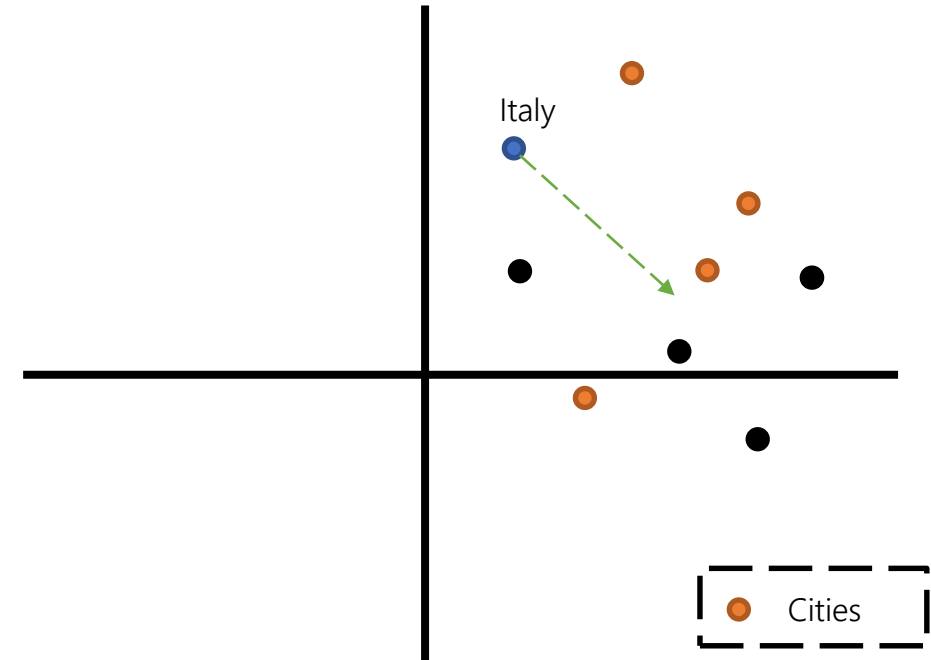
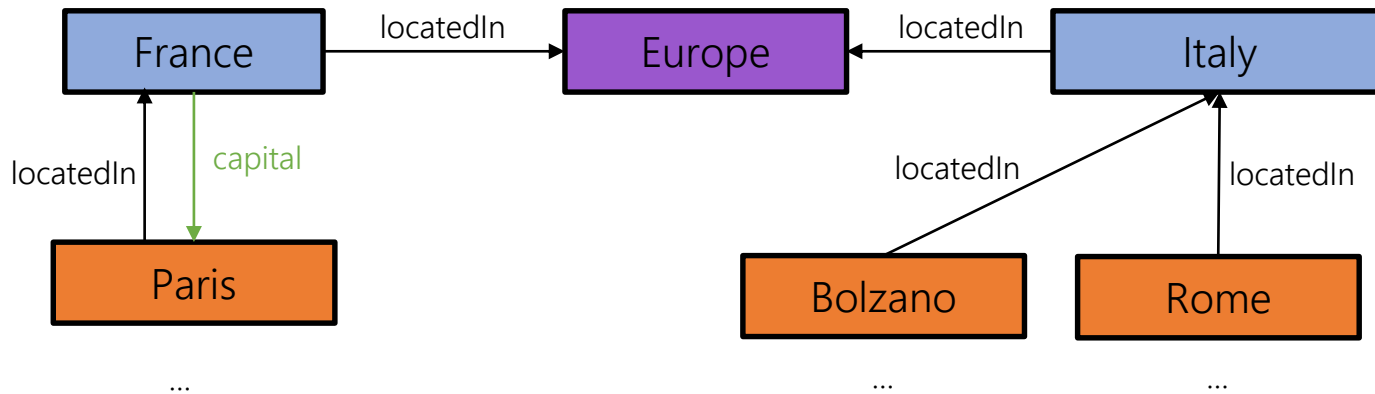
Knowledge Graph Completion



Link Prediction: Direct Applications

Generating embeddings allows to use well-know **machine learning** models over KG (e.g., classification, clustering,...). However, we can also use them directly:

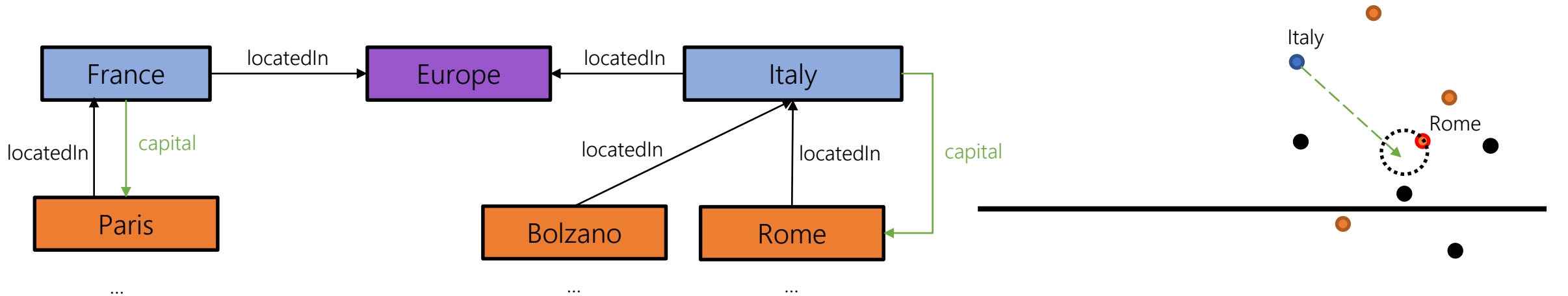
Knowledge Graph Completion



Link Prediction: Direct Applications

Generating embeddings allows to use well-know **machine learning** models over KG (e.g., classification, clustering,...). However, we can also use them directly:

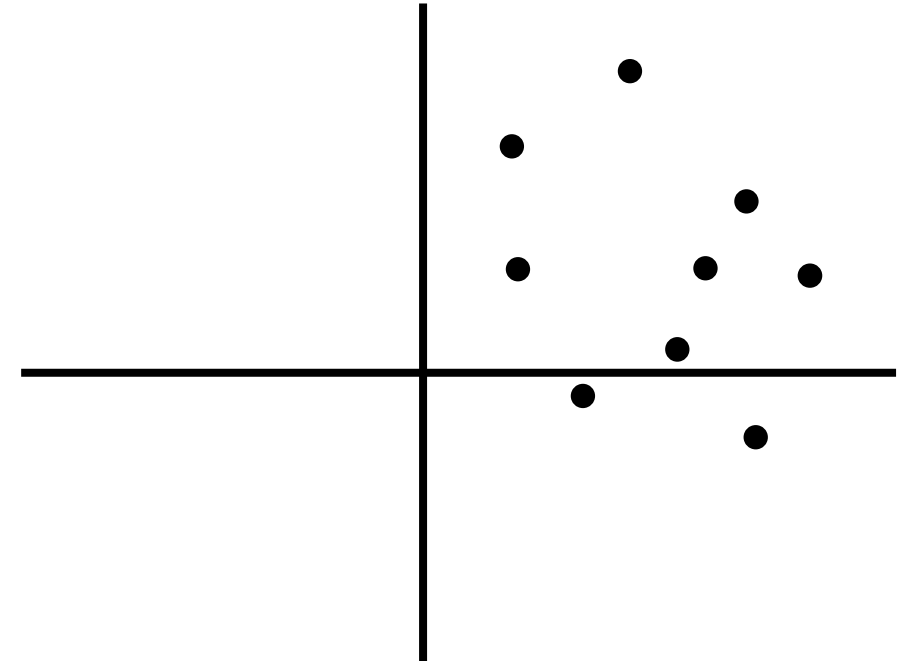
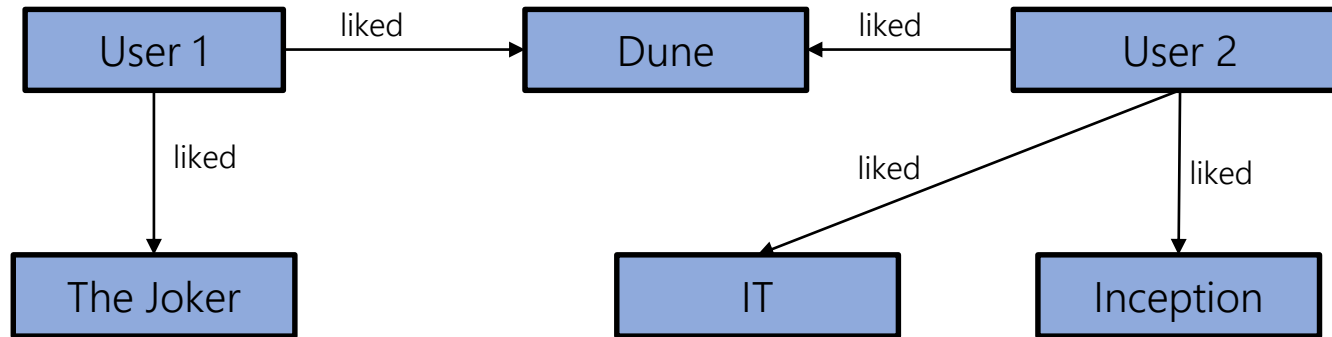
Knowledge Graph Completion



Link Prediction: Direct Applications

Generating embeddings allows to use well-know **machine learning** models over KG (e.g., classification, clustering,...). However, we can also use them directly:

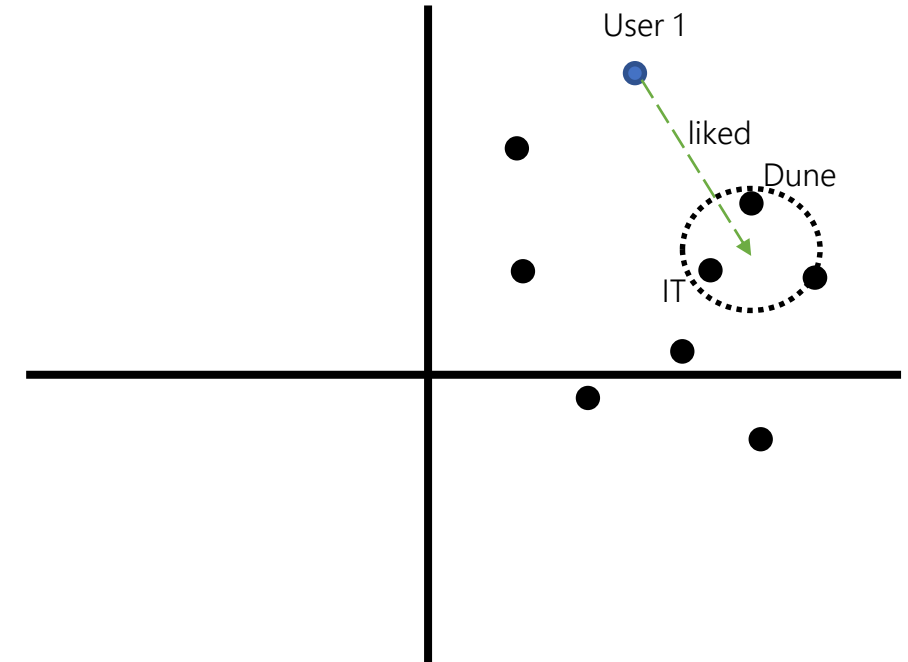
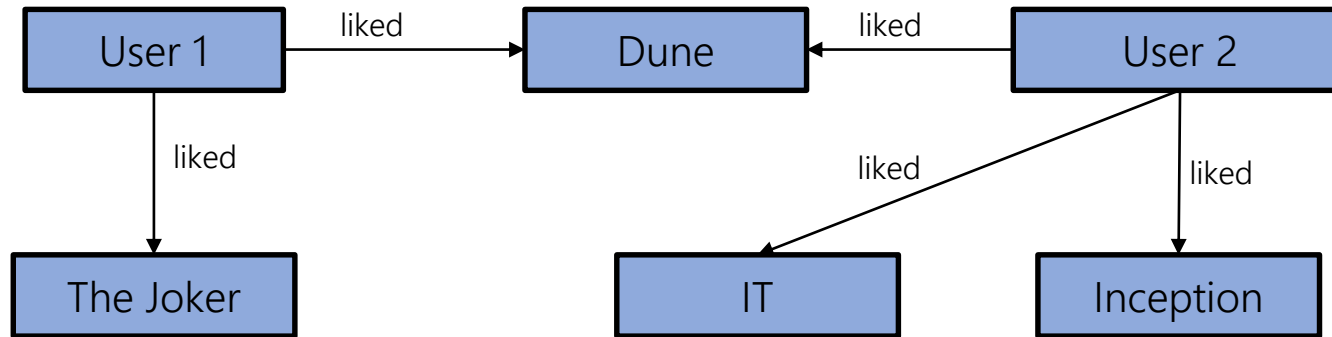
Recommender Systems



Link Prediction: Direct Applications

Generating embeddings allows to use well-know **machine learning** models over KG (e.g., classification, clustering,...). However, we can also use them directly:

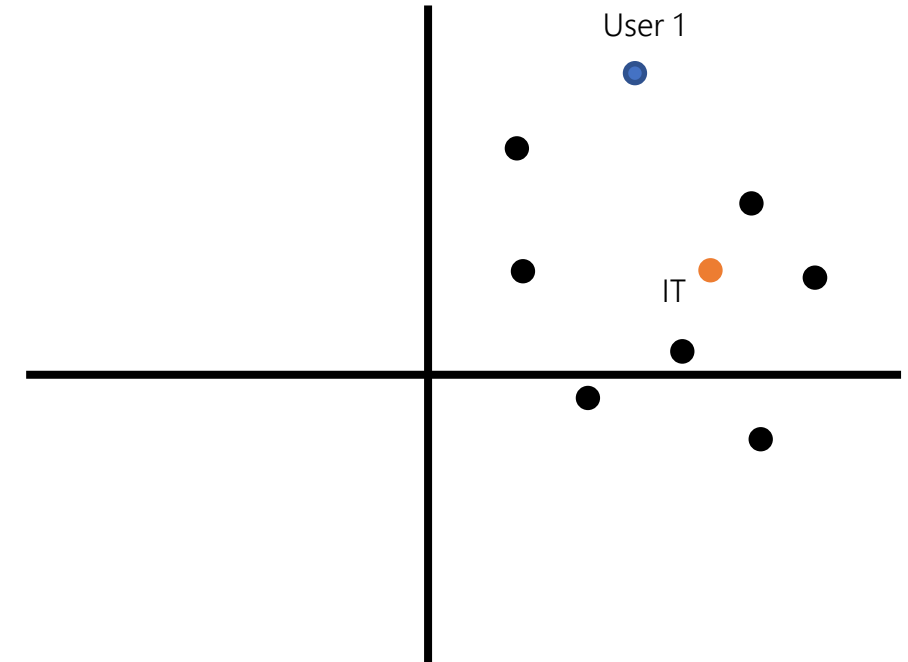
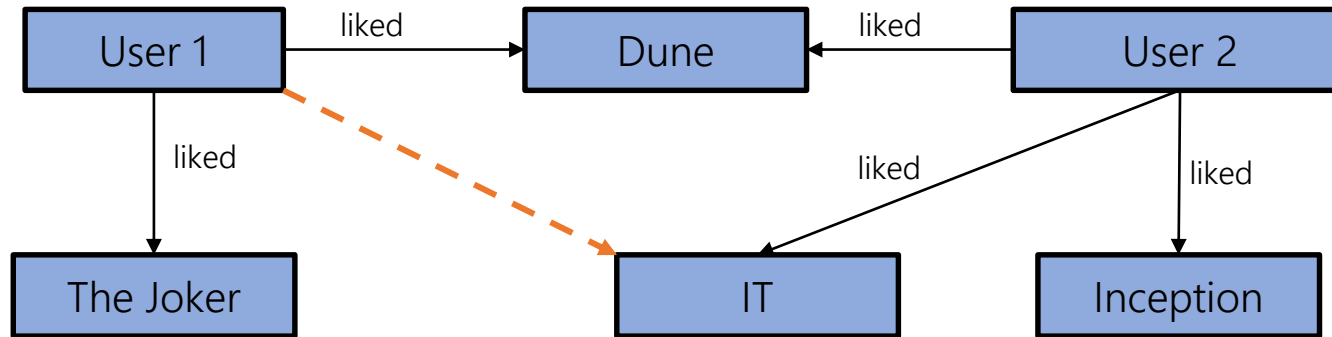
Recommender Systems



Link Prediction: Direct Applications

Generating embeddings allows to use well-know **machine learning** models over KG (e.g., classification, clustering,...). However, we can also use them directly:

Recommender Systems



Questions?

